EDURA - Learning Management System

Software Design Specification (SDS) - Final Version

By

W.G.N.DANANJAYA
SE/2021/047
R.W.V.I.D.RAJAPAKSHA
SE/2021/029
P.D.D.RANSIKA
SE/2021/034
H.T.MADUSHANKA
SE/2021/011

A report submitted in partial fulfillment of the requirements for the degree of Bachelor of Science Honors in Software Engineering (B.Sc. SE)

Software Engineering Teaching Unit

Faculty of Science

University of Kelaniya

Sri Lanka

2026

VERSION HISTORY

Version	Date	Author(s)	Changes
1.0	19/04/2026	Nadun Dananjaya, Imansha Dilshan, Danindu Ransika, Thushan Madhusanka	Initial SDS document creation
2.0	25/10/2026	Nadun Dananjaya, Imansha Dilshan, Danindu Ransika, Thushan Madhusanka	Final SDS. Design updated to the system as implemented: architecture, database design, class and sequence diagrams, security design and deployment design revised; changes recorded in Chapter 9; entity-relationship diagrams added as Appendix A.

TABLE OF CONTENTS

LIST OF TABLES v
LIST OF FIGURES vi
CHAPTER 1 INTRODUCTION 1
1.1 Document Purpose 1
1.2 System Overview 1
1.3 Scope of This Document 1
CHAPTER 2 ARCHITECTURAL DESIGN 3
2.1 High-Level Architecture: Microservices 3
2.2 Cloud-Native Architecture 5
2.3 Justification for Architectural Choices 6
2.3.1 Why Microservices? 6
2.3.2 Why Nginx as API Gateway? 6
2.3.3 Why RabbitMQ for Message Broker? 6
2.3.4 Why Redis for Caching? 7
2.3.5 Why PostgreSQL Neon? 7
2.4 Scalability & Maintainability Considerations 7
CHAPTER 3 DATABASE DESIGN 8
3.1 Database Architecture Overview 8
3.2 Entity-Relationship Diagram 8
3.3 Database Normalization Strategy 8
CHAPTER 4 CLASS DIAGRAM 10
4.1 Class Diagram Overview 10
4.2 Class Diagram 10
CHAPTER 5 SEQUENCE DIAGRAMS 11
5.1 Sequence Diagram Overview 11
5.2.2 Course Creation and Publishing Flow 12
5.2.3 Student Enrollment and Payment Processing Flow 13
5.2.4 Assessment Taking and Automated Grading Flow 14
5.2.5 Progress Tracking and Certificate Generation Flow 14
5.2.6 Analytics Report Generation Flow 15
5.2.7 Content Streaming Flow 15

CHAPTER 6 UI/UX DESIGN 16
6.1 Wireframes for Key User Interfaces 16
6.1.1 Authentication Interfaces 16
6.1.2 Student Dashboard 17
6.1.3 Course Catalog/Browse Interface 18
6.1.4 Course Detail and Lesson List 18
6.1.5 Assessment Interface 19
6.1.6 Progress and Analytics Dashboard 20
6.1.7 Admin Dashboard 21
6.1.8 Profile and Settings 21
6.2 User Flow Diagrams 22
6.3 Accessibility Considerations (WCAG 2.1 Level AA) 28
CHAPTER 7 SECURITY DESIGN 30
7.1 Security Architecture Overview 30
7.2 Authentication Mechanism 30
7.2.1 Authentication Service and Google Sign-In 30
7.2.2 Token Management 31
7.2.3 Session Management 32
7.2.4 Email Verification and Password Reset 32
7.3 Authorization Levels 34
7.3.1 Role-Based Access Control (RBAC) 34
7.3.2 Permission Enforcement 36
7.4 Data Protection Approach 37
7.4.1 Data Classification 37
7.4.2 Data Storage Security 37
7.4.3 Data Transmission Security 38
7.5 Input Validation and Sanitization Strategy 39
7.5.1 Client-Side Validation 39
7.5.2 Server-Side Validation 39
7.5.3 File Upload Validation 40
7.6 Encryption Standards 41
7.6.1 Data at Rest Encryption 41
7.6.2 Data in Transit Encryption 41

7.6.3 Encryption Key Management 42
CHAPTER 8 DEPLOYMENT DESIGN 43
8.1 Hosting Plan & Infrastructure 43
8.2 Hardware & Resource Requirements 43
8.3 Network Diagram 44
8.4 Environment Specifications 44
8.5 CI/CD Pipeline Overview 45
CHAPTER 9 CHANGES FROM VERSION 1.0 45
CHAPTER 10 REFERENCES 49
APPENDIX A ENTITY-RELATIONSHIP DIAGRAMS 50
A.1 Auth Service 50
A.2 User Service 51
A.3 Course Service 52
A.4 Content Service 53
A.5 Enrollment Service 54
A.6 Payment Service 55
A.7 Assessment Service 56
A.8 Progress Service 57
A.9 Notification Service 57
A.10 Analytics Service 58
A.11 Admin Service 59

LIST OF TABLES

Table 1: Hardware and Resource Requirements 43
Table 2: Changes from Version 1.0 45

LIST OF FIGURES

Figure 1: High-Level Microservice Architecture Diagram 5
Figure 2: Entity - Relationship Diagram 8
Figure 3: Class Diagram 10
Figure 4: User Authentication and Login Flow Diagram..... 11
Figure 5: Course Creation and Publishing Flow Diagram 12
Figure 6: Student Enrollment and Payment Processing Flow Diagram 13
Figure 7: Assessment Taking and Automated Grading Flow Diagram 14
Figure 8: Progress Tracking and Certificate Generation Flow Diagram 14
Figure 9: Analytics Report Generation Flow Diagram 15
Figure 10: Content Streaming Flow 15
Figure 11: User Sign-in Interface 16
Figure 12: User Sign-up Interface 17
Figure 13: Student Dashboard Interface 17
Figure 14: Course Catalog Interface 18
Figure 15: Course Detail Interface 18
Figure 16: Course Lesson List Interface (locked until enrolment) 19
Figure 17: Assessment Interface 19
Figure 18: Admin Analytics: Summary Statistics 20
Figure 19: Admin Analytics: Charts 20
Figure 20: Admin Dashboard Interface 21
Figure 21: User Profile Interface 21
Figure 22: Student Registration and Onboarding Flow 22
Figure 23: Course Enrollment Flow 23
Figure 24: Learning Path Flow 24
Figure 25: Assessment Flow 25
Figure 26: Teacher Course Creation Flow 26
Figure 27: Admin Payment Verification Flow 27
Figure 28: Authentication Flow 31
Figure 29: Registration Flow 33
Figure 30: Password Reset Flow 34
Figure 31: Network diagram 44

CHAPTER 1 INTRODUCTION

1.1 Document Purpose

The purpose of this System Design Specification (SDS) document is to provide a comprehensive technical design blueprint for the EDURA Learning Management System. This final version (2.0) describes the system as it was implemented: it outlines the architecture, database schema, class structures, interaction flows, user interface specifications, security mechanisms and deployment strategy, and Chapter 9 lists every change made since version 1.0.

1.2 System Overview

EDURA is a comprehensive web-based Learning Management System designed specifically for the ICT - Dimuthu educational institute. The system facilitates online education for Advanced Level A/L students by enabling educators to create, manage, and deliver digital courses, assessments, and learning materials.

The platform employs a microservices architecture to ensure scalability, maintainability and independent deployment of services. The system is built using Next.js 15 with React 19 and Redux Toolkit for the frontend, eleven Python (FastAPI) microservices for the backend, PostgreSQL on Neon for data persistence (one database per service), Redis for sessions and caching, and RabbitMQ for asynchronous communication between the services. Authentication is implemented by the Auth Service (email and password with JWT access and refresh tokens, and optional Google sign-in), media is delivered through Cloudinary and YouTube, payments are supported through bank-deposit receipts verified by administrators and through the PayHere gateway (implemented on its sandbox and hidden in the checkout page), and email is sent through the Resend API.

1.3 Scope of This Document

This document covers the technical design aspects of the EDURA system, including:

- High-level and detailed architectural design with justification
- Database schema design with normalization strategies
- Object-oriented design through class diagrams
- System behavior through sequence diagrams

- User interface specifications and accessibility considerations
- Security architecture includes
- authentication, authorization, and data protection
- Deployment architecture on Microsoft Azure Kubernetes Service with GitHub Actions CI/CD pipelines

CHAPTER 2 ARCHITECTURAL DESIGN

2.1 High-Level Architecture: Microservices

The system is decomposed into loosely coupled, independently deployable services, each responsible for a specific business capability. The services communicate asynchronously through events on a RabbitMQ topic exchange; they do not call one another over HTTP, and each service keeps small read-only replicas of the data it needs.

Key Architectural Components:

1. **API Gateway (Nginx)**
    - Entry point for all client requests
    - Request routing to appropriate microservices
    - Rate limiting (100 requests per minute per client address, burst of 20)
    - CORS policy and a 25 MB request body limit; HTTPS is terminated by the Caddy edge proxy in front of the gateway
    - Token validation is performed by each service (shared/auth.py), not by the gateway
2. **Microservices Layer**
    - User Service
    - Authentication Service
    - Course Service
    - Content Service
    - Enrollment Service
    - Assessment Service
    - Progress Tracking Service
    - Payment Service
    - Notification Service
    - Analytics Service

- Admin Service

3. **Message Broker (RabbitMQ)**
    - Asynchronous inter-service communication
    - Event-driven architecture support
    - Decoupling of services
    - Message persistence and reliability
4. **Cache Layer (Redis)**
    - Session management
    - Frequently accessed data caching
    - Performance optimization
    - Reduced database load
5. **Data Persistence Layer (PostgreSQL Neon)**
    - Distributed database per service pattern
    - Data isolation and independence
    - ACID compliance for transactional operations
6. **External Services**
    - Google (optional sign-in)
    - Cloudinary (Media storage and streaming)
    - PayHere (Payment gateway, sandbox)
    - YouTube (private lecture video hosting)
    - Resend (transactional email)

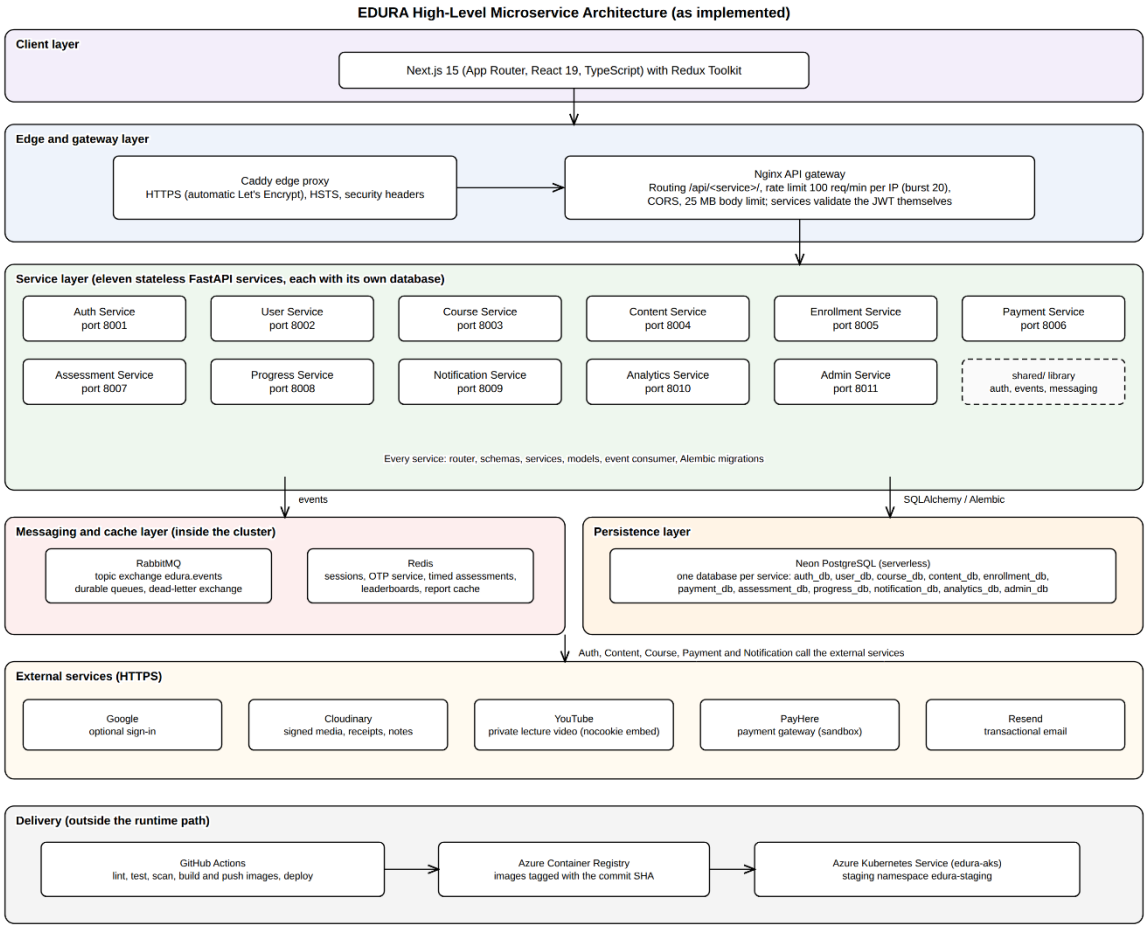


Figure 1: High-Level Microservice Architecture Diagram

2.2 Cloud-Native Architecture

The system utilizes **Microsoft Azure** to handle infrastructure management.

- **Container Orchestration: Azure Kubernetes Service (AKS)** manages the lifecycle of the microservices, handling self-healing and service discovery.
- **Global Content Delivery:** Media and video streaming are offloaded to **Cloudinary's CDN** to ensure low-latency video playback for students across the country.
- **Authentication:** implemented by the Auth Service (email and password, JWT access and refresh tokens) with optional Google sign-in. Asgardeo was not used (Chapter 9).
- **Serverless Scaling: PostgreSQL Neon** provides a serverless database layer that scales compute resources up or down automatically based on active query load.

2.3 Justification for Architectural Choices

2.3.1 Why Microservices?

Scalability: Individual services can be scaled independently based on load. For example, during assessment periods, the Assessment Service can be scaled horizontally without affecting other services.

Technology Flexibility: Different services can use optimal technology stacks. While Python is used for backend services, the choice allows for service-specific optimizations.

Independent Deployment: Services can be deployed, updated, and rolled back independently, reducing deployment risk and downtime.

Team Autonomy: Development teams can work on different services simultaneously without coordination overhead.

Fault Isolation: Failure in one service does not cascade to the entire system: services communicate through events and keep local replicas, so a peer being unavailable does not stop them. Circuit breakers were not implemented.

Business Alignment: Each service maps to a specific business capability, making the system easier to understand and maintain.

2.3.2 Why Nginx as API Gateway?

- High performance and low resource consumption
- Proven load balancing capabilities
- Rate limiting, CORS and path-based routing to the frontend and the services
- Cost-effective compared to commercial alternatives

2.3.3 Why RabbitMQ for Message Broker?

- Robust message persistence and delivery guarantees
- Support for multiple messaging patterns (pub/sub, point-to-point, request/reply)
- Easy integration with Python services
- Proven reliability in production environments

2.3.4 Why Redis for Caching?

- In-memory data structure store provides exceptional read/write performance
- Support for complex data structures (strings, hashes, lists, sets)
- Pub/Sub capabilities for real-time features
- Native support in Python ecosystem

2.3.5 Why PostgreSQL Neon?

- Serverless PostgreSQL with automatic scaling
- Separation of storage and compute
- Built-in connection pooling and caching
- Branching capabilities for development/testing

2.4 Scalability & Maintainability Considerations

- **Horizontal Scalability:** All services are **stateless**, allowing AKS to spin up additional pods during high-traffic periods (like A/L exam seasons).
- **Independent Scaling:** Resources can be allocated specifically to high-load services like "Content Streaming" without over-provisioning the "User Service".
- **CI/CD Maintainability:** GitHub Actions pipelines run linting, tests, dependency scans and a migration check on every push, deploy develop to staging automatically and require manual approval before production.
- **Observability:** Health is monitored through readiness, liveness and startup probes and post-deployment smoke tests. Azure Monitor and centralised logging were not implemented.

CHAPTER 3 DATABASE DESIGN

3.1 Database Architecture Overview

The EDURA system employs a **database-per-service pattern** where each microservice maintains its own dedicated database schema. This approach ensures data isolation, independent scaling, and service autonomy. PostgreSQL Neon is utilized as the database management system across all services due to its serverless architecture, automatic scaling capabilities, and PostgreSQL compatibility.

3.2 Entity-Relationship Diagram

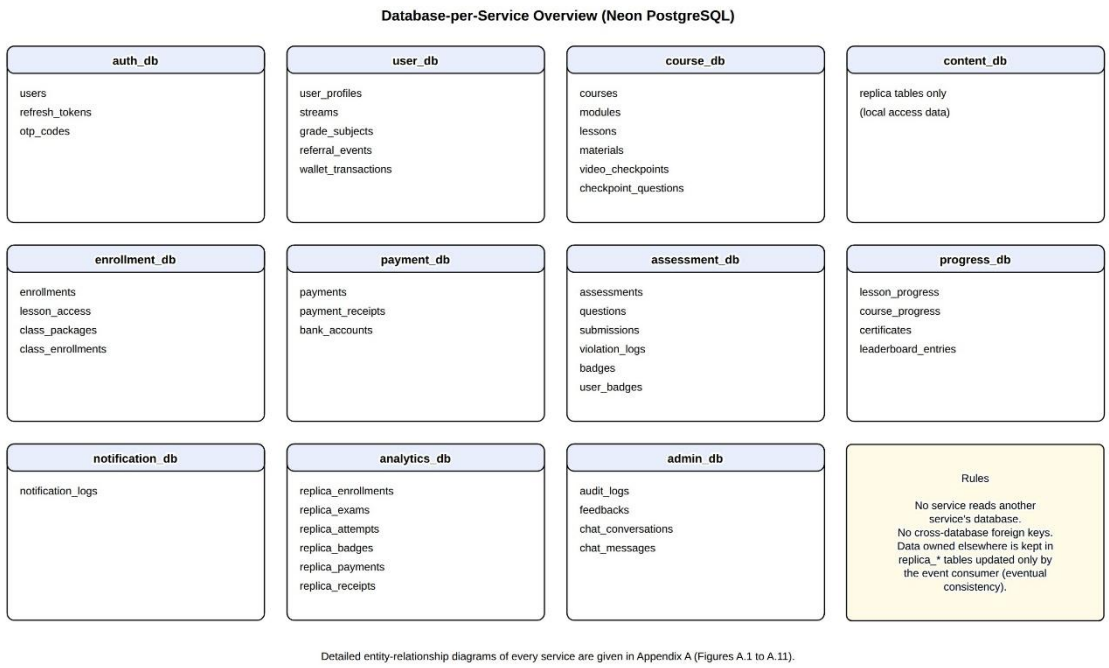


Figure 2: Entity - Relationship Diagram

3.3 Database Normalization Strategy

All database schemas are designed following **Third Normal Form (3NF)** principles [1] to ensure:

- **Data integrity**: Elimination of redundancy and update anomalies
- **Consistency**: Single source of truth for each data element
- **Efficiency**: Optimized storage and query performance
- **Maintainability**: Easier schema evolution and updates

Normalization Process:

1. **First Normal Form (1NF)**: All tables have atomic values and no repeating groups
2. **Second Normal Form (2NF)**: All non-key attributes are fully functionally dependent on the primary key
3. **Third Normal Form (3NF)**: All non-key attributes are independent of other non-key attributes

Denormalization Considerations:

Strategic denormalization is applied in specific cases for read-heavy operations:

- Read-only replica tables (replica_*) updated from events, so that services never query each other's databases
- Cached summary analytics and leaderboards in Redis
- Session data in Redis

CHAPTER 4 CLASS DIAGRAM

4.1 Class Diagram Overview

The EDURA system follows object-oriented design principles with a layered structure that is the same in every microservice: a router (presentation layer), Pydantic schemas (validation), services (business logic), SQLAlchemy models (domain entities), an event consumer and Alembic migrations. Cross-cutting classes and functions are held in a shared package. The figure shows the principal domain classes and the Auth Service classes.

4.2 Class Diagram

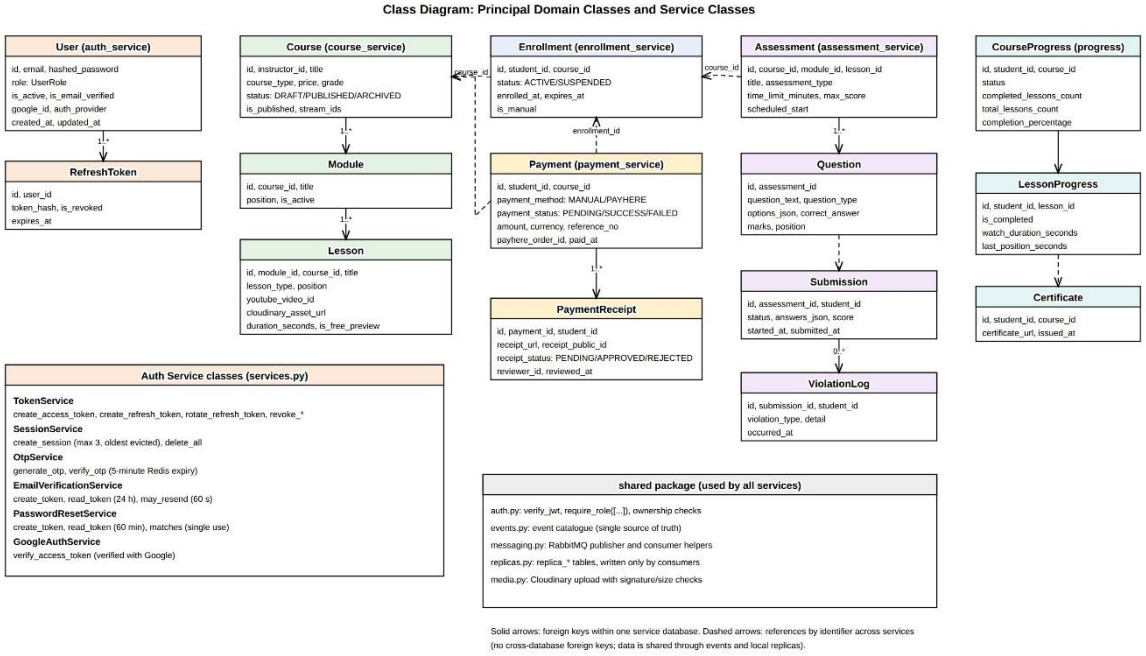


Figure 3: Class Diagram

CHAPTER 5 SEQUENCE DIAGRAMS

5.1 Sequence Diagram Overview

Sequence diagrams illustrate the temporal ordering of interactions between system components for specific use cases. The following diagrams describe the principal flows as they were implemented. The AI assistant flow of version 1.0 (Gemini) was not implemented and is not shown.

5.2 Sequence Diagrams

5.2.1 User Authentication and Login Flow

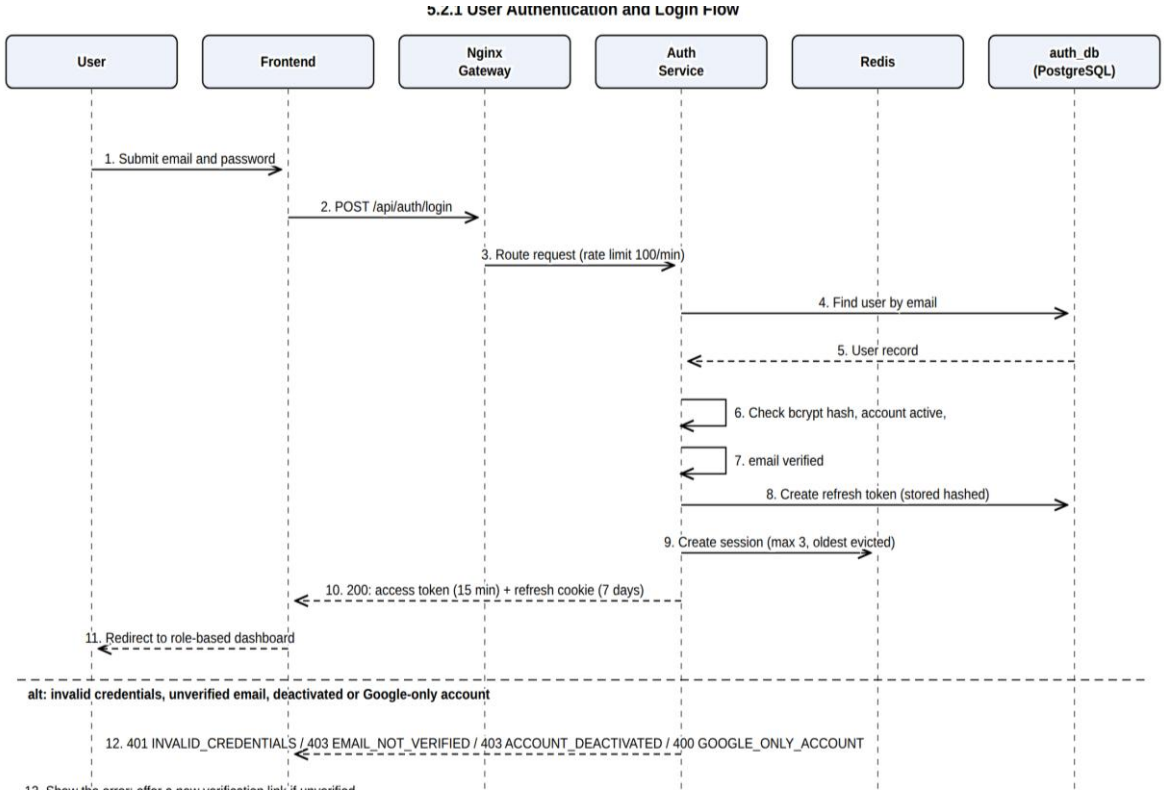


Figure 4: User Authentication and Login Flow Diagram

5.2.2 Course Creation and Publishing Flow

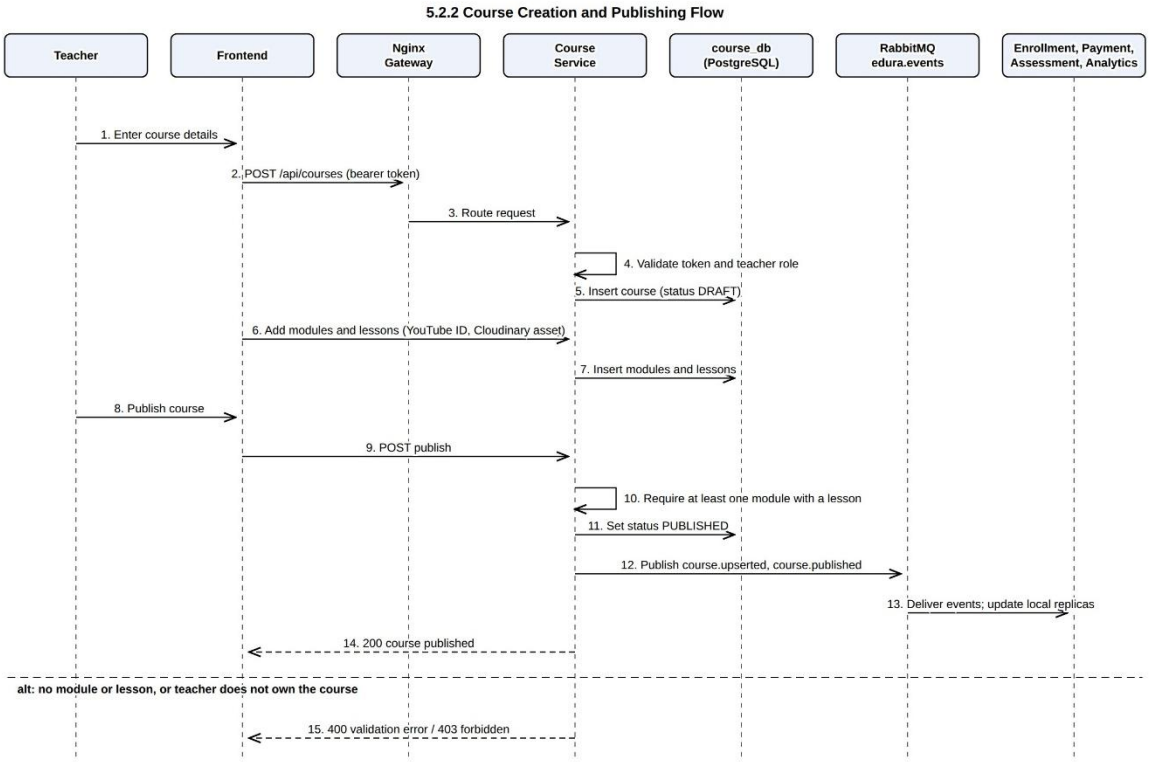


Figure 5: Course Creation and Publishing Flow Diagram

5.2.3 Student Enrollment and Payment Processing Flow

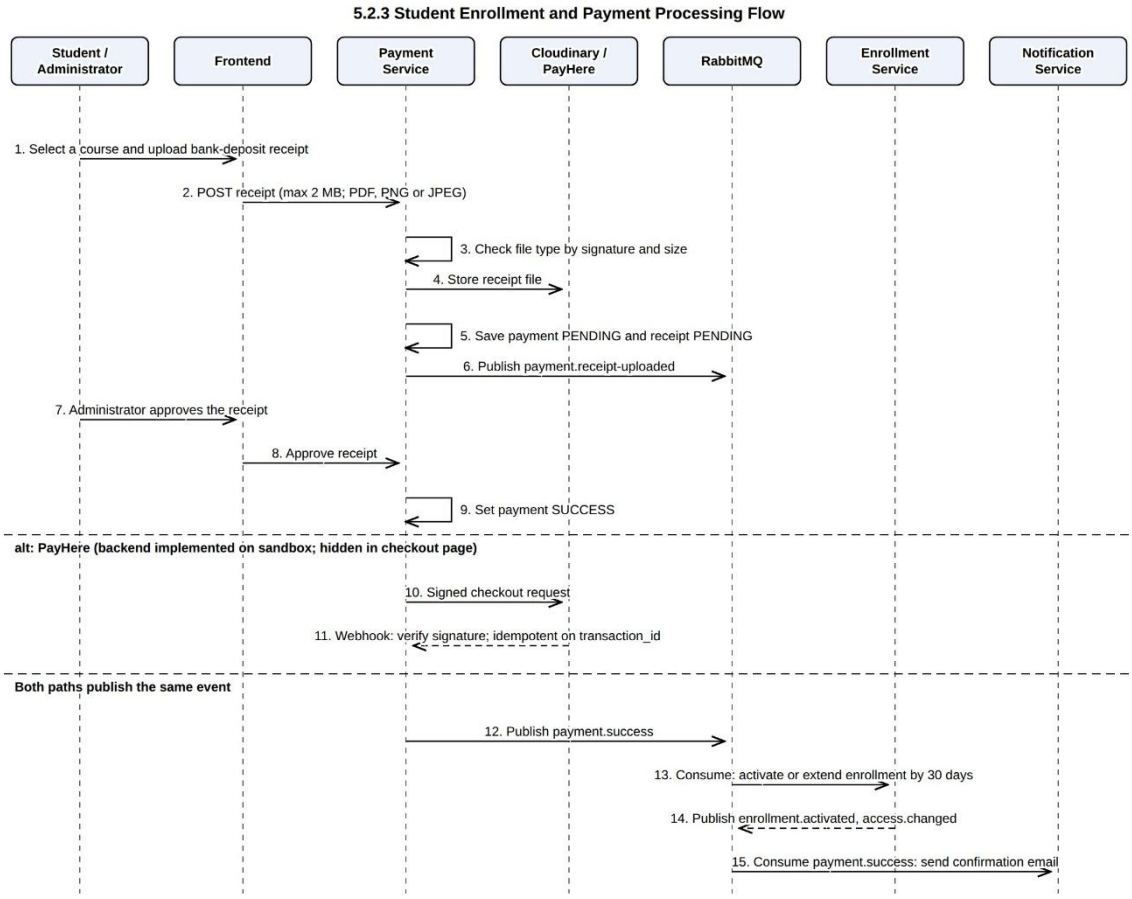


Figure 6: Student Enrollment and Payment Processing Flow Diagram

5.2.4 Assessment Taking and Automated Grading Flow

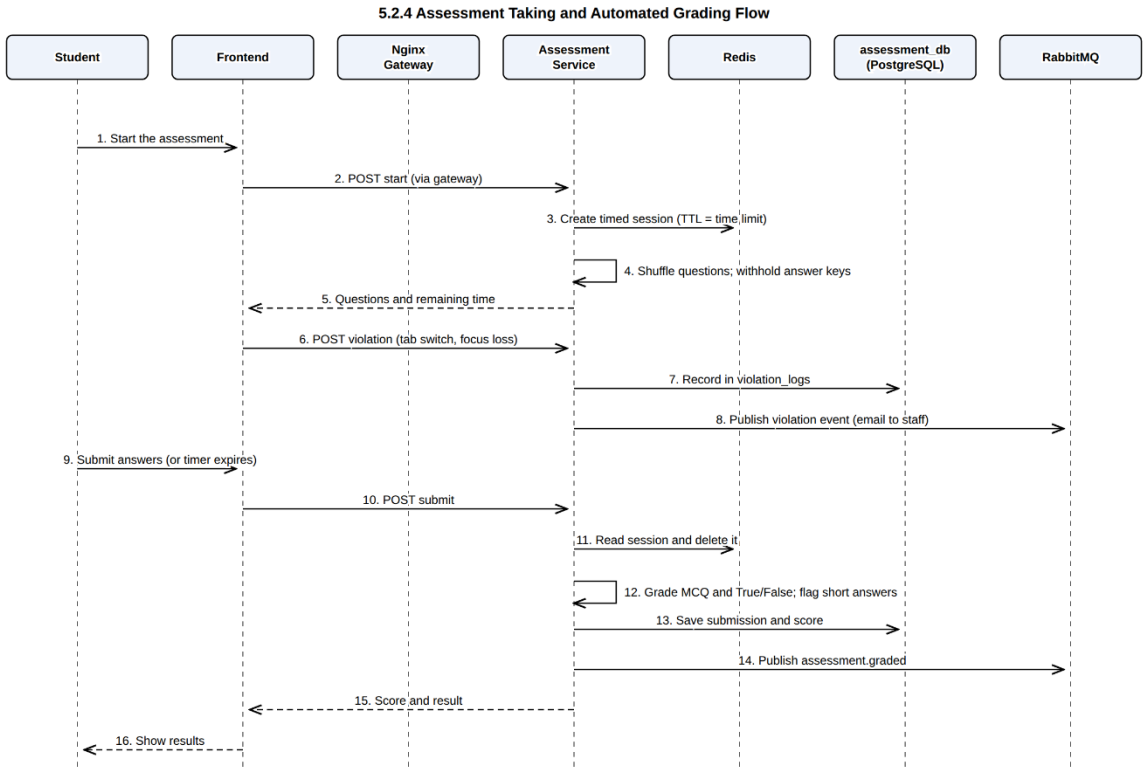


Figure 7: Assessment Taking and Automated Grading Flow Diagram

5.2.5 Progress Tracking and Certificate Generation Flow

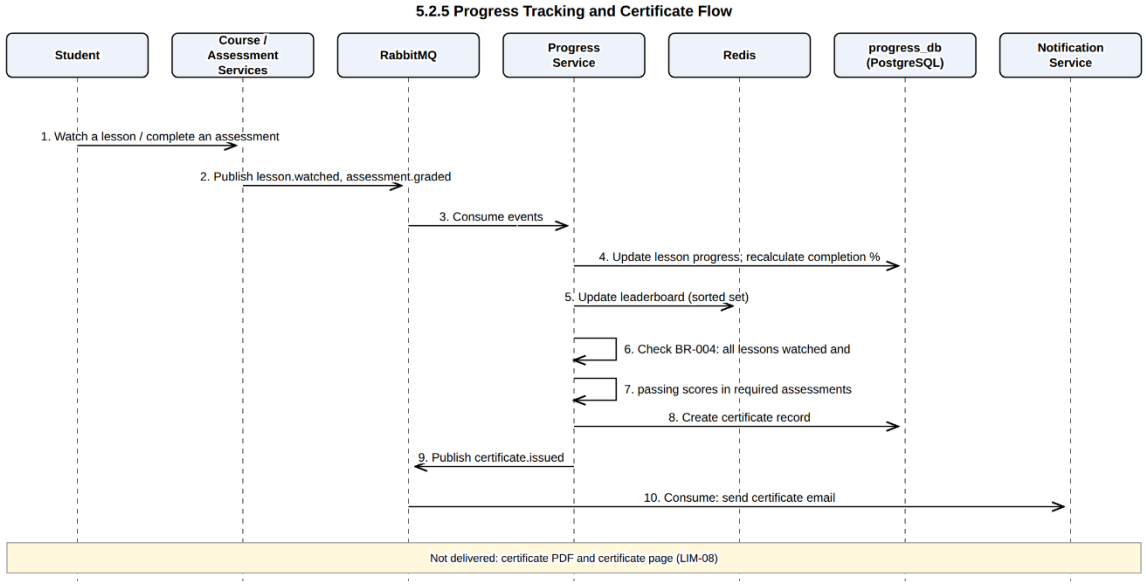


Figure 8: Progress Tracking and Certificate Generation Flow Diagram

5.2.6 Analytics Report Generation Flow

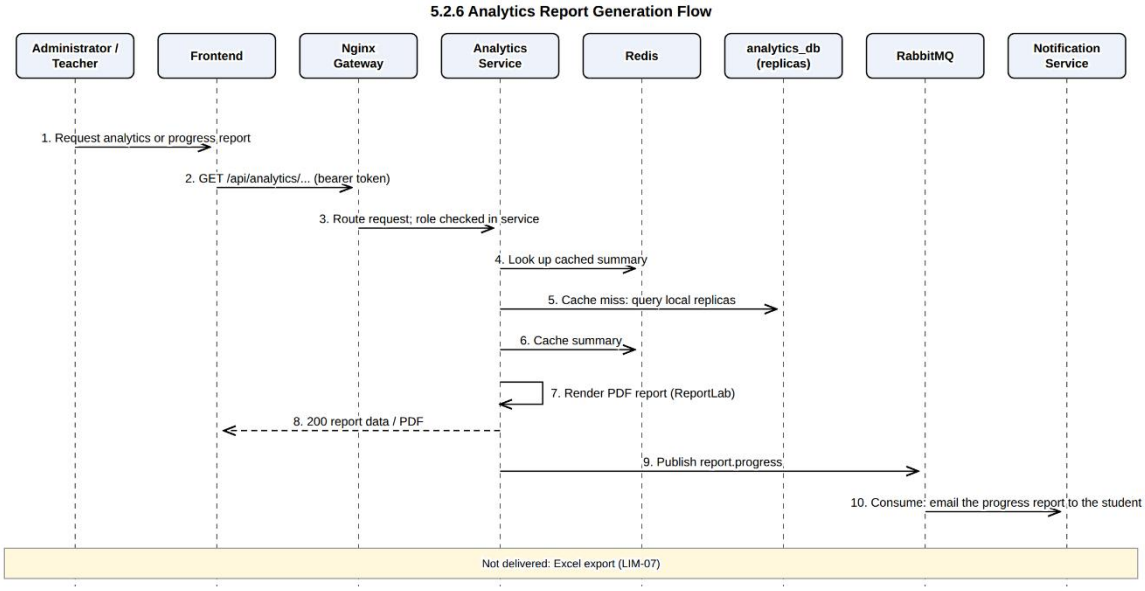


Figure 9: Analytics Report Generation Flow Diagram

5.2.7 Content Streaming Flow

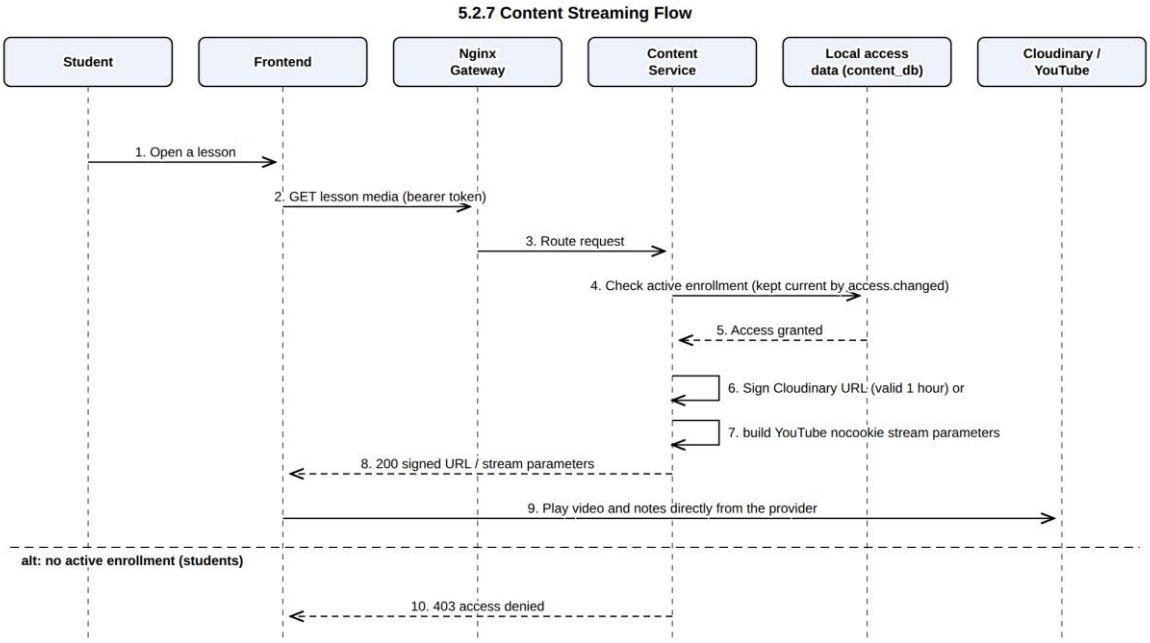


Figure 10: Content Streaming Flow

CHAPTER 6 UI/UX DESIGN

6.1 Wireframes for Key User Interfaces

Figures 12 to 17 and 19 to 22 are screenshots of the implemented pages taken on the staging environment (staging.eduralms.com) with test accounts; the e-mail address of the student account is replaced by a placeholder. The student has no enrolment, so the course page shows its lessons as locked; the lesson player and the exam screens appear only after enrolment and are not shown. Figure 18 is the wireframe prepared before development; it was used as the specification of the Next.js frontend, and the implemented exam screen differs from it in detail. The implemented route structure is: public and authentication pages (landing page, information pages, login, register, verify-email, forgot-password, reset-password); student pages (courses, video-courses, video-classes, class-packages, my-courses, exams, my-results, rankings, profile with badges, feedback, and payment pages for checkout and receipt upload); administrator pages (dashboard, analytics, courses, students, admins, payments, class-packages, streams, subjects, rankings, messages, settings).

6.1.1 Authentication Interfaces

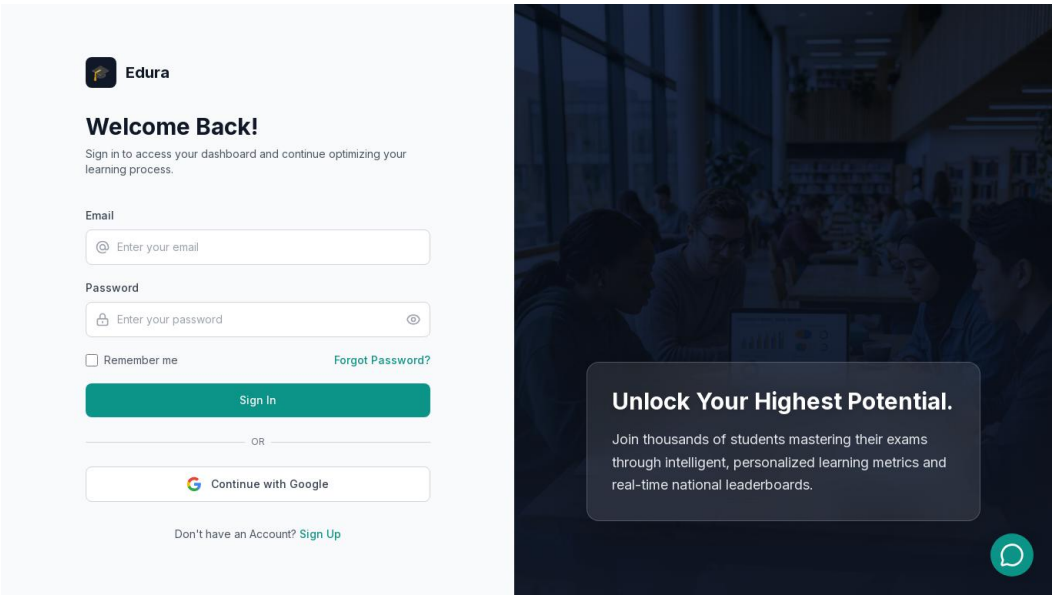


Figure 11: User Sign-in Interface

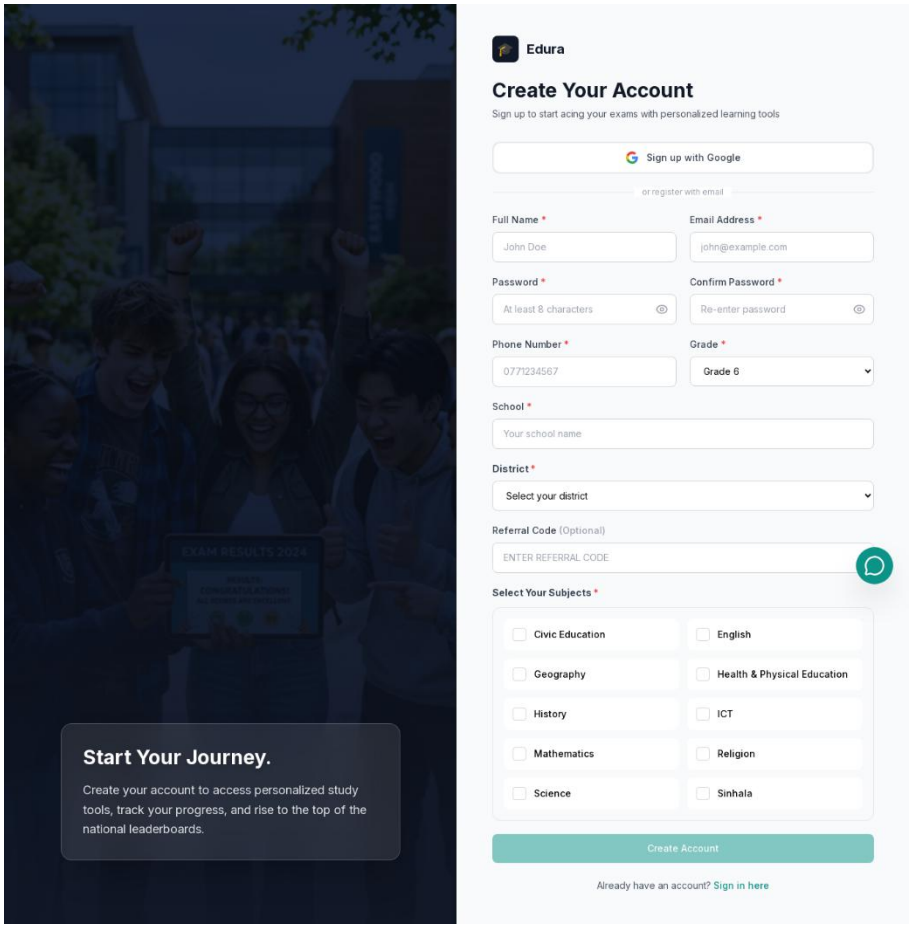


Figure 12: User Sign-up Interface

6.1.2 Student Dashboard

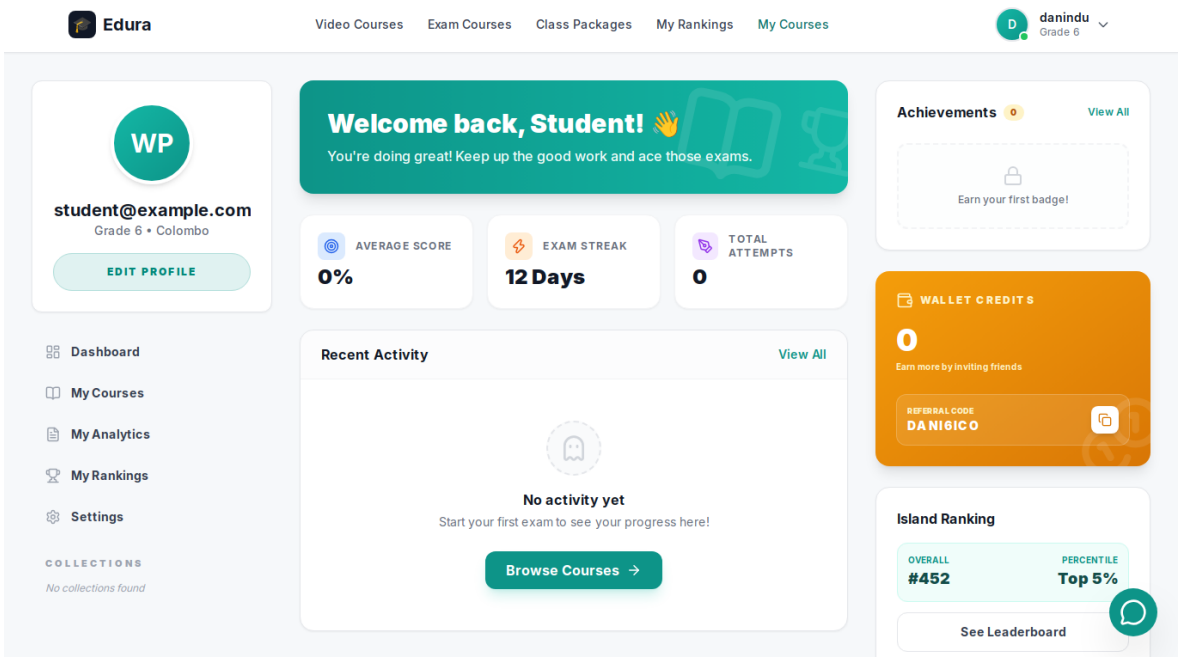


Figure 13: Student Dashboard Interface

6.1.3 Course Catalog/Browse Interface

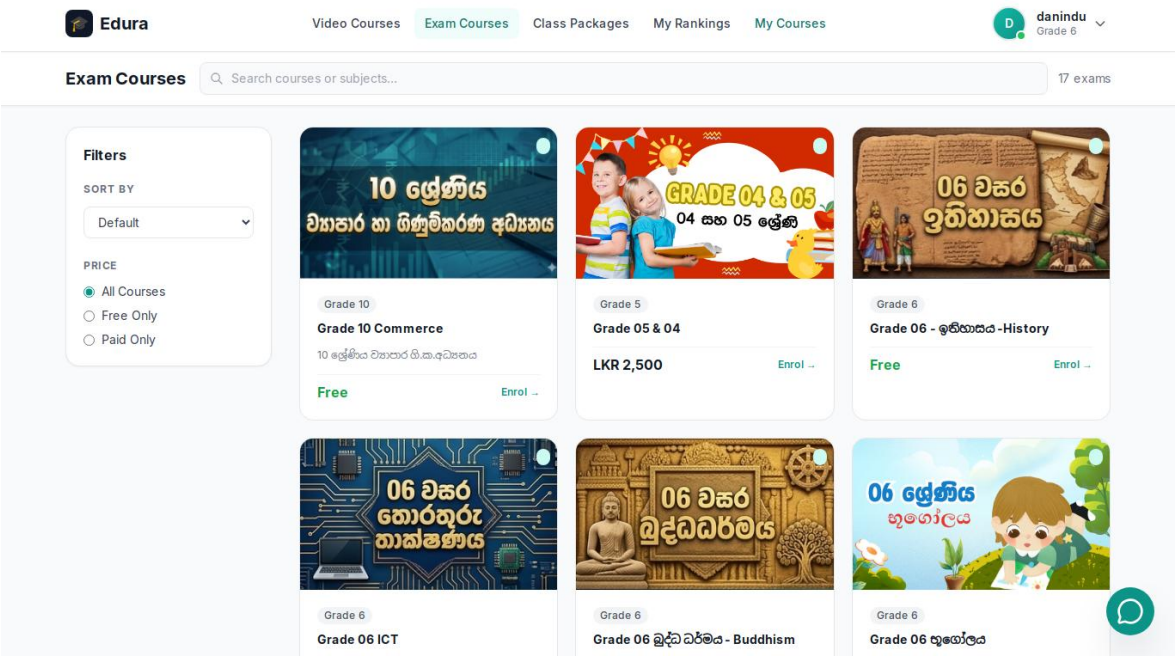


Figure 14: Course Catalog Interface

6.1.4 Course Detail and Lesson List

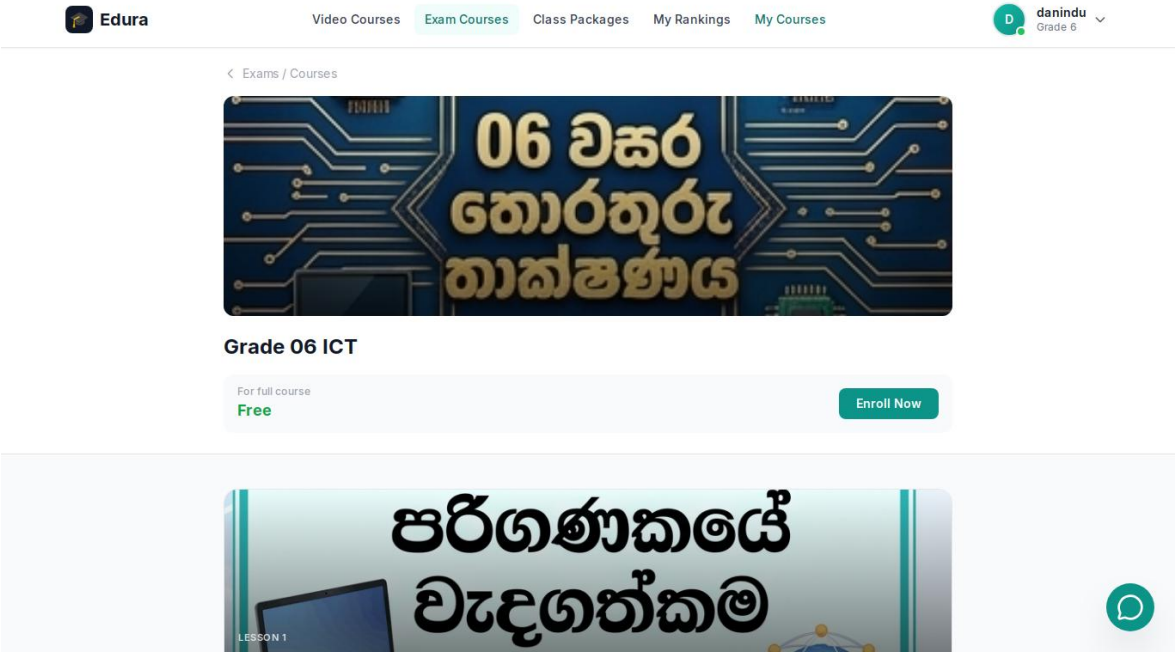


Figure 15: Course Detail Interface

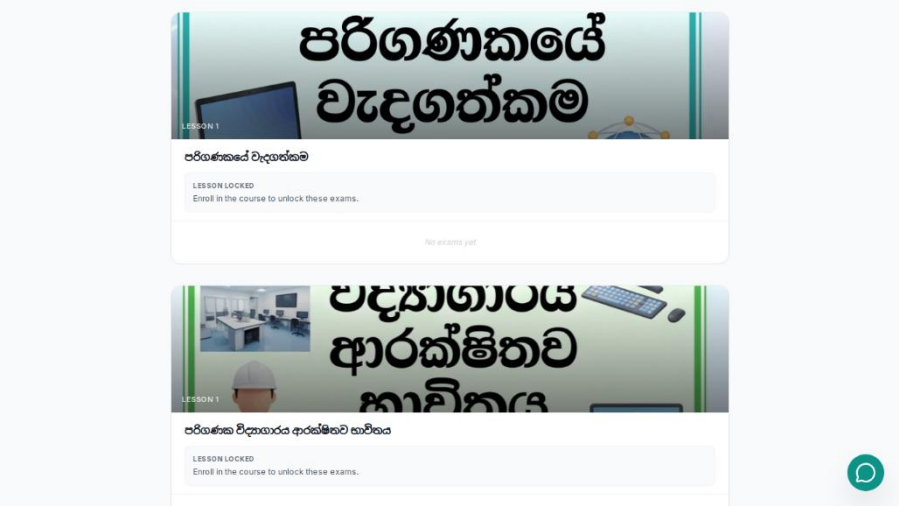


Figure 16: Course Lesson List Interface (locked until enrolment)

6.1.5 Assessment Interface

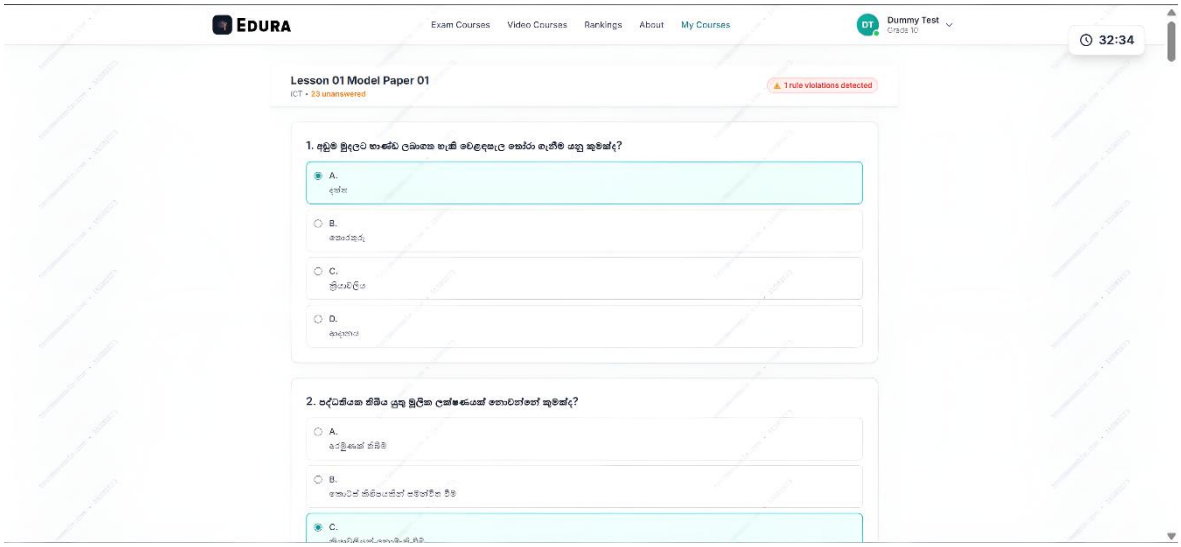


Figure 17: Assessment Interface

6.1.6 Progress and Analytics Dashboard

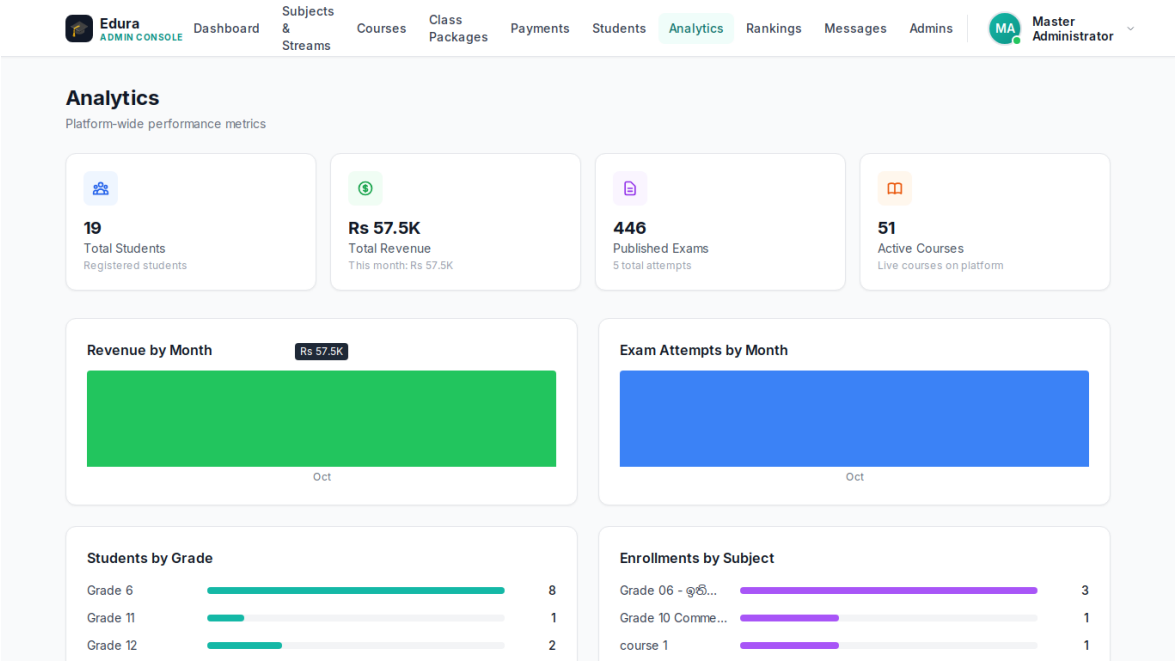


Figure 18: Admin Analytics: Summary Statistics

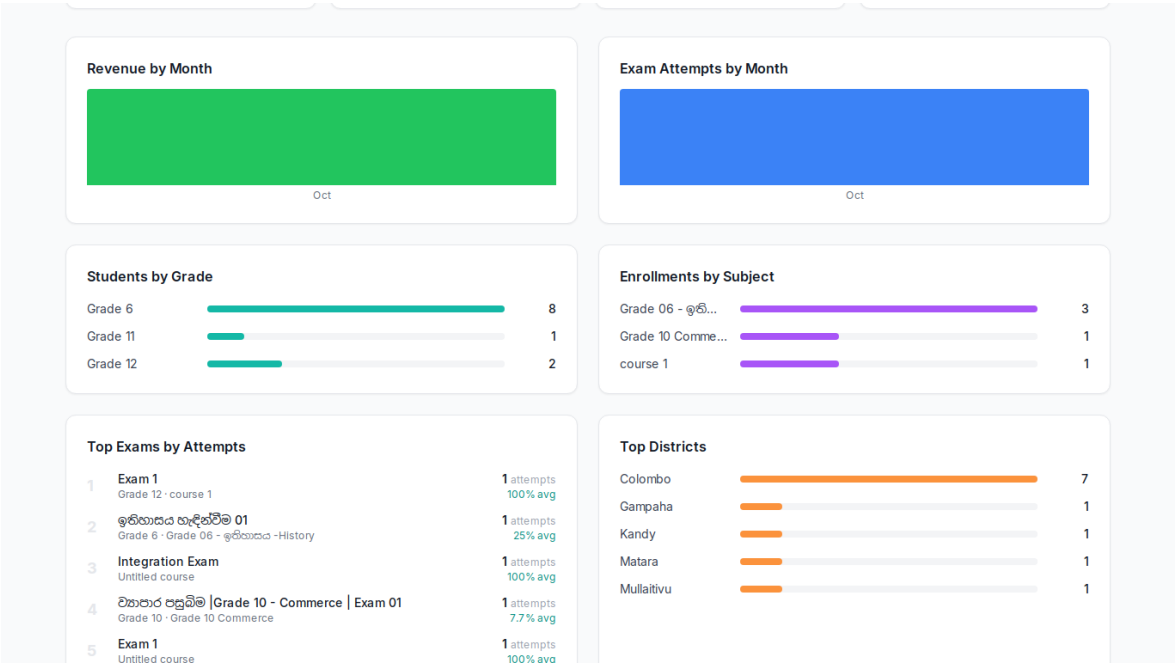


Figure 19: Admin Analytics: Charts

6.1.7 Admin Dashboard

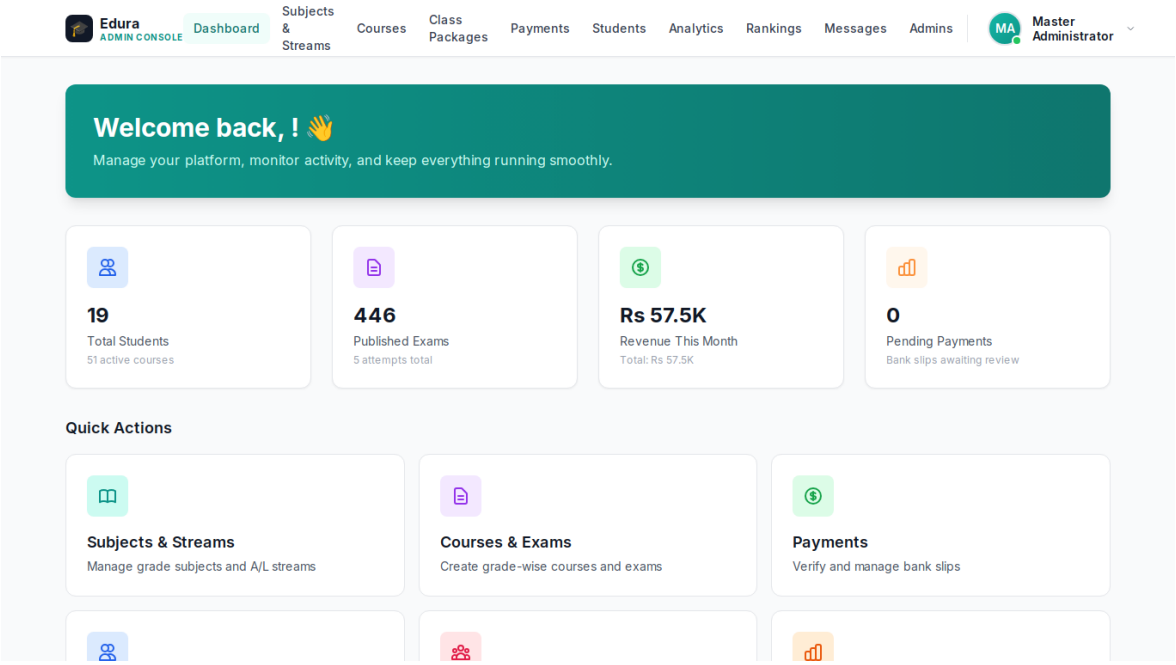


Figure 20: Admin Dashboard Interface

6.1.8 Profile and Settings

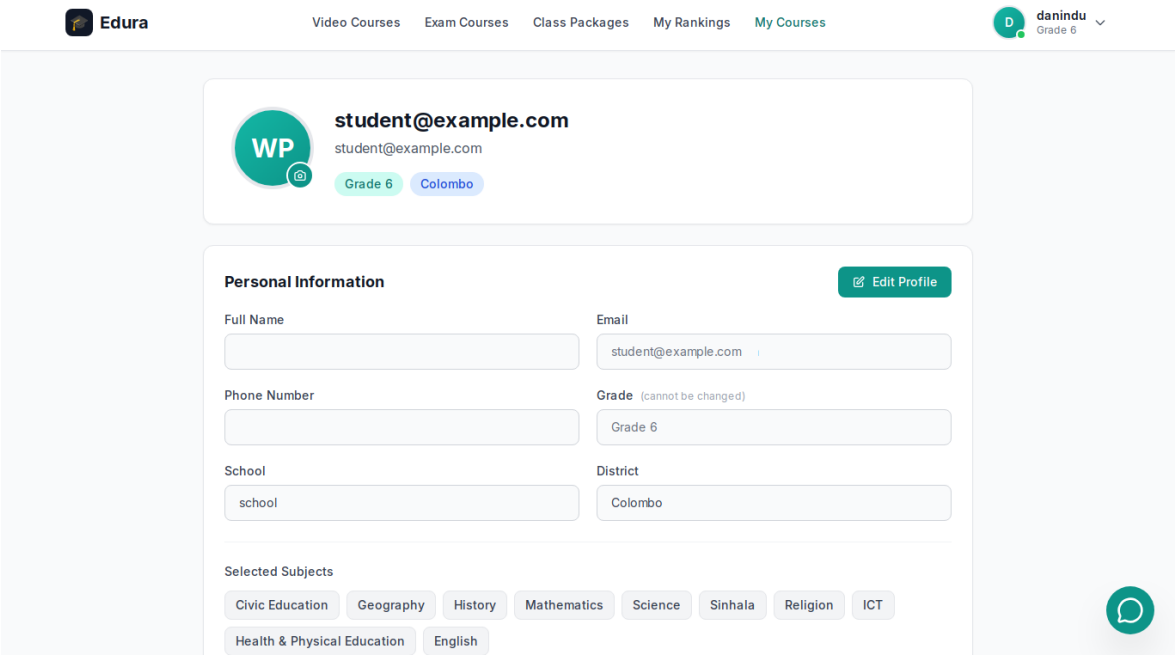


Figure 21: User Profile Interface

6.2 User Flow Diagrams

The registration and onboarding flow has been redrawn to match the implemented registration and verification. The other user flows in this section are the flows prepared at design stage; in the implemented system the PayHere option is hidden in the checkout page (receipt upload with administrator verification is the active payment route) and no certificate page exists.

1. Student Registration and Onboarding Flow

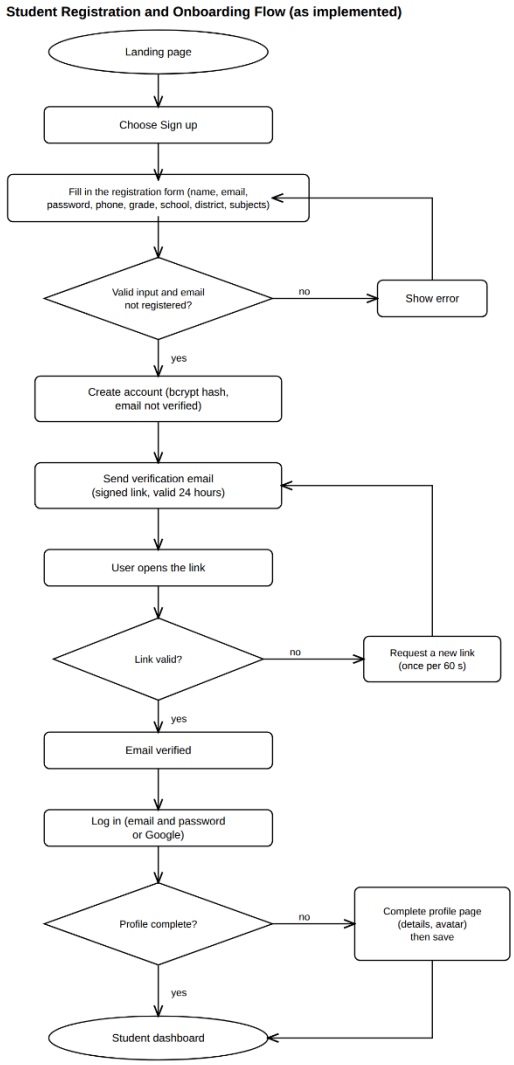


Figure 22: Student Registration and Onboarding Flow

2. Course Enrollment Flow

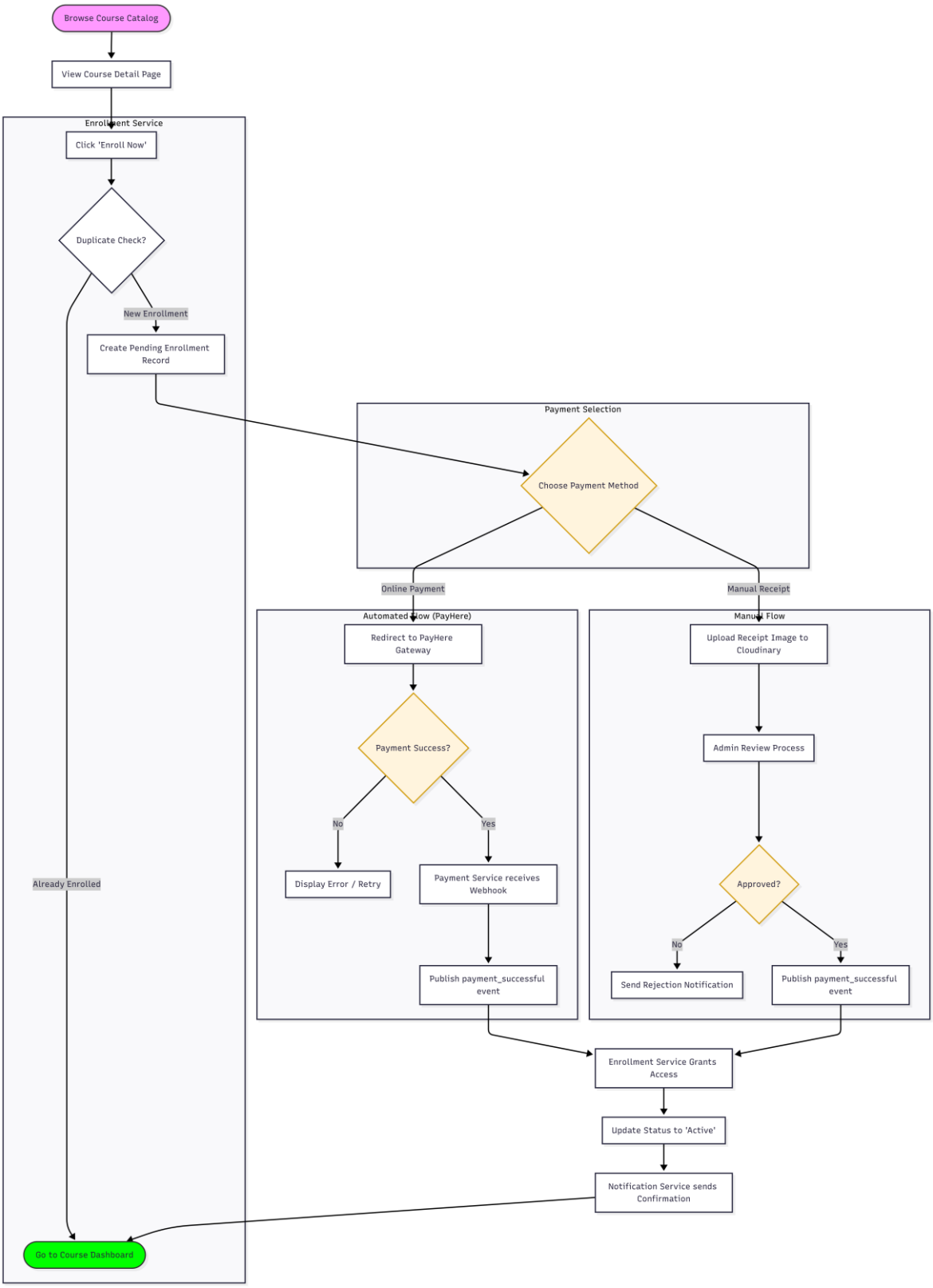


Figure 23: Course Enrollment Flow

3. Learning Path Flow

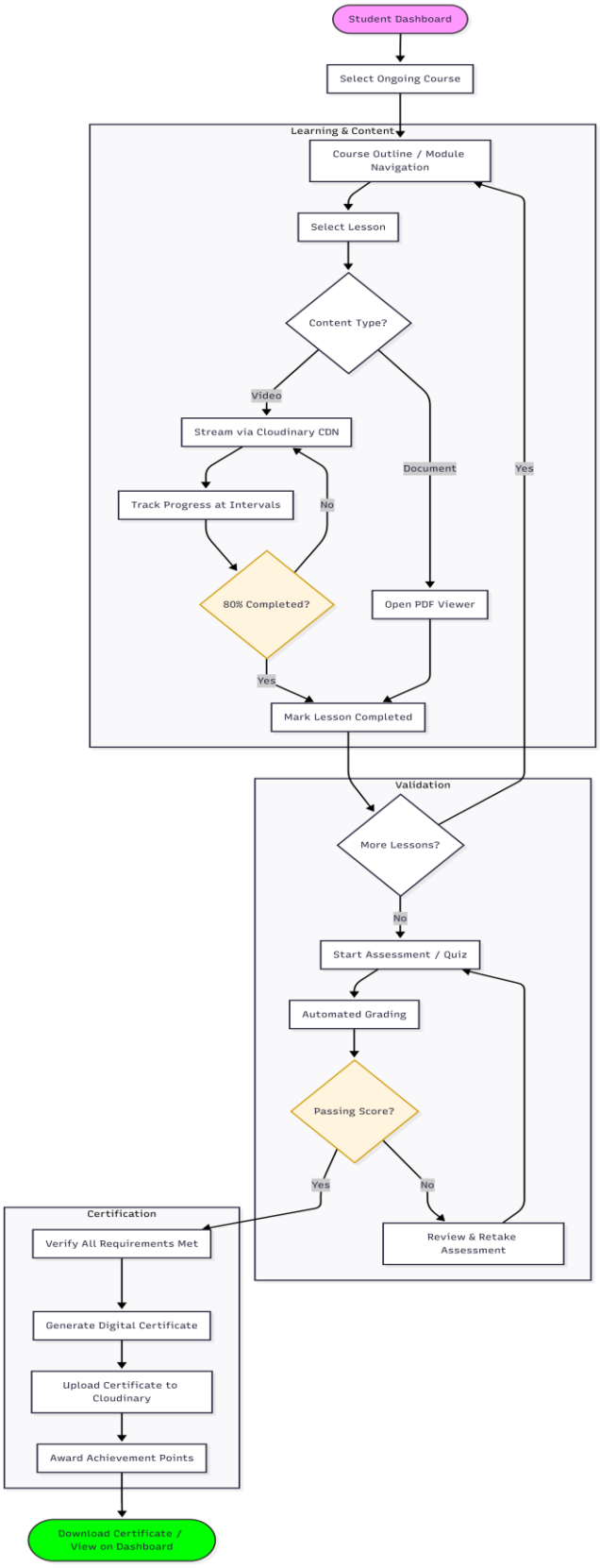


Figure 24: Learning Path Flow

4. Assessment Flow

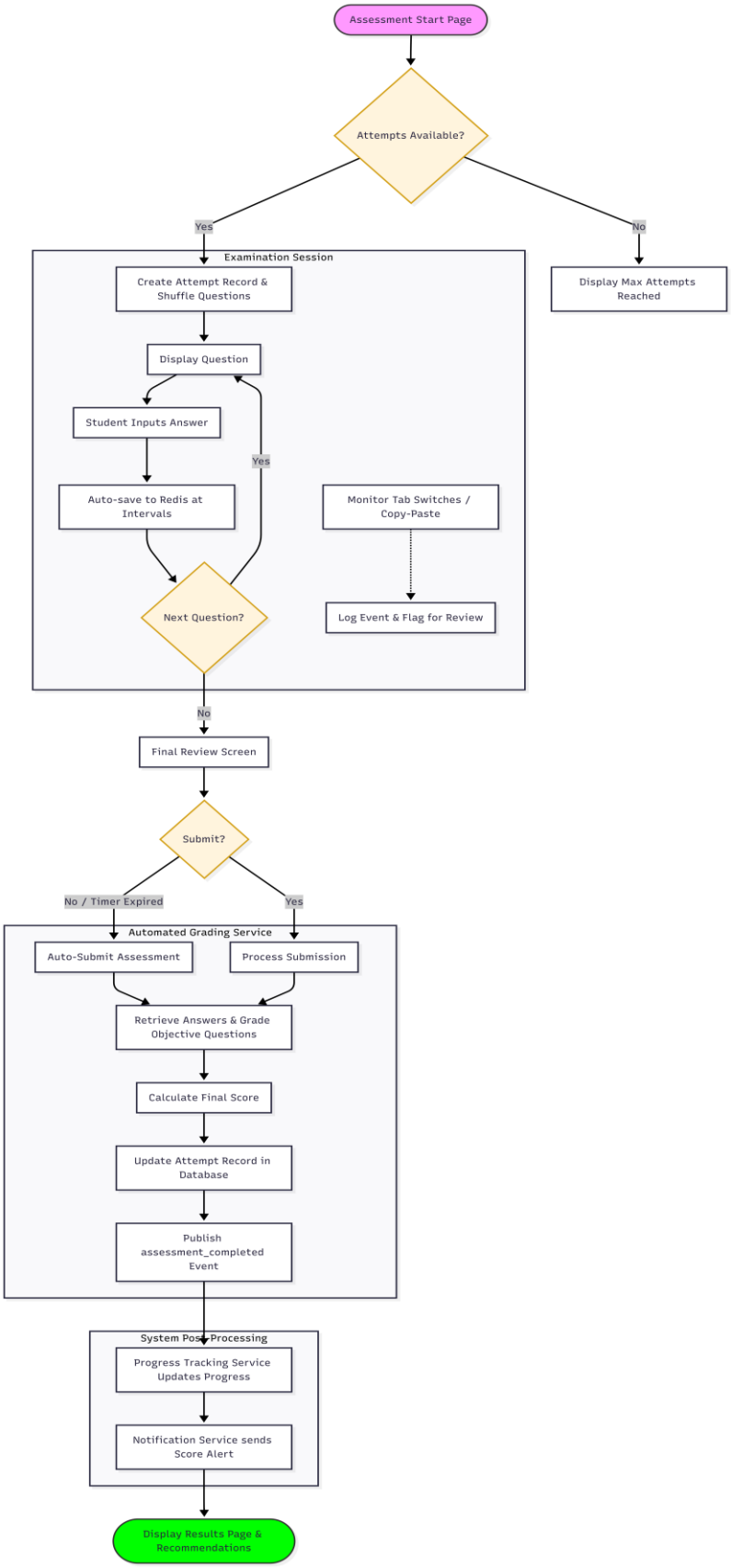


Figure 25: Assessment Flow

5. Teacher Course Creation Flow

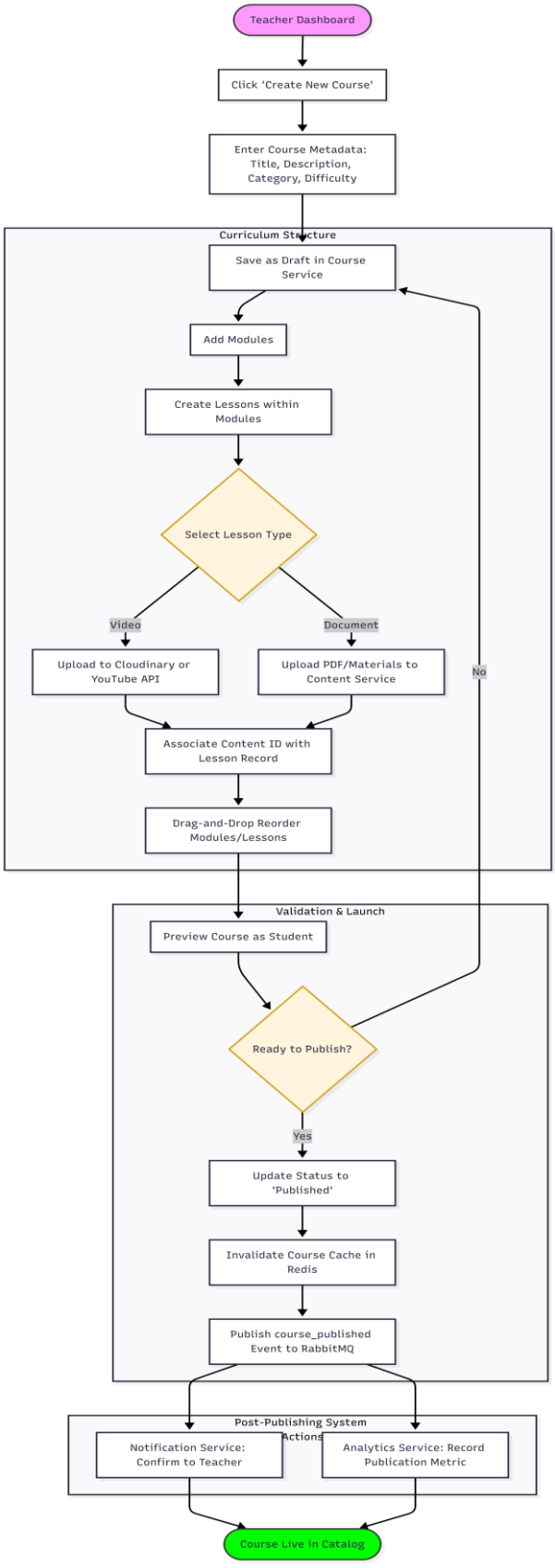


Figure 26: Teacher Course Creation Flow

6. Admin Payment Verification Flow

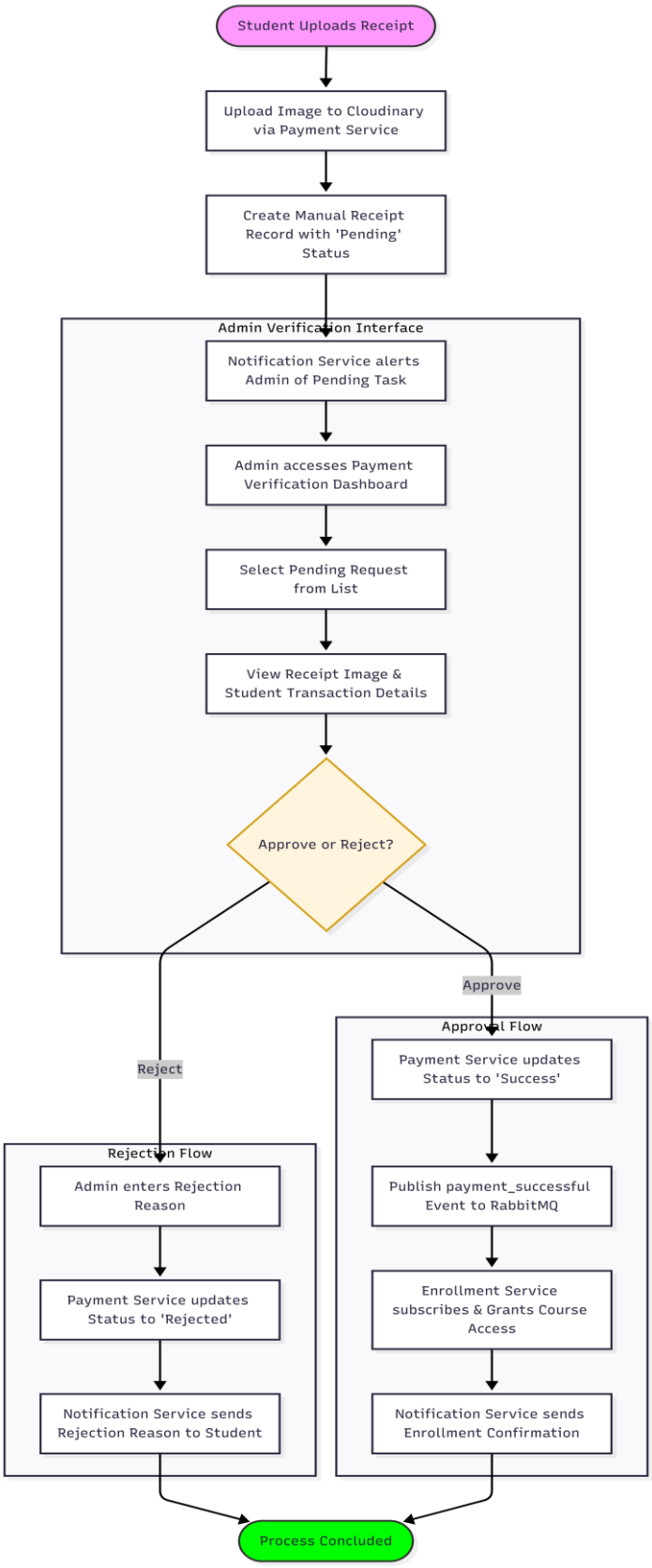


Figure 27: Admin Payment Verification Flow

6.3 Accessibility Considerations (WCAG 2.1 Level AA)

The items below are design targets. Formal accessibility testing was not carried out, and small-screen verification is recorded as a known limitation.

Perceivable:

- All images have alt text
- Videos have captions and transcripts
- Color is not the only means of conveying information
- Text contrast ratio of at least 4.5:1
- Resizable text up to 200% without loss of functionality

Operable:

- All functionality available via keyboard
- No keyboard traps
- Skip navigation links
- Clearly visible focus indicators
- Sufficient time for timed assessments with extension option
- Pause, stop, hide options for auto-updating content

Understandable:

- Clear and consistent navigation
- Error identification and suggestions
- Form labels and instructions
- Predictable behavior
- Help documentation available

Robust:

- Valid HTML semantic markup
- ARIA labels and roles where appropriate

- Compatible with assistive technologies
- Screen-reader testing was not performed

CHAPTER 7 SECURITY DESIGN

7.1 Security Architecture Overview

The EDURA platform implements a comprehensive, defense-in-depth security strategy across all layers of the application stack. Security is integrated into every component through authentication, authorization, encryption, input validation, and continuous monitoring.

7.2 Authentication Mechanism

7.2.1 Authentication Service and Google Sign-In

Authentication Service:

- The Auth Service implements registration, login, token issue and rotation, Google sign-in, email verification and password reset
- Email and password authentication with bcrypt-hashed passwords; Google sign-in uses an OAuth 2.0 [3] access token that the Auth Service verifies with Google
- Multi-factor authentication is not implemented
- User identities are stored in auth_db; extended profile data is stored in user_db
- Single sign-on with external identity systems is not supported

Authentication Flow:

1. User submits email and password to the EDURA frontend
2. The frontend sends POST /api/auth/login through the gateway to the Auth Service
3. The Auth Service checks the bcrypt hash, that the account is active and that the email is verified
4. On success it issues a 15-minute access token (HS256 JWT) and a 7-day refresh token
5. The refresh token is set in an HttpOnly cookie and stored hashed in the database; a Redis session is created
6. The access token is sent as a bearer token on later API requests and validated by each service
7. POST /api/auth/refresh rotates the refresh token and issues a new access token

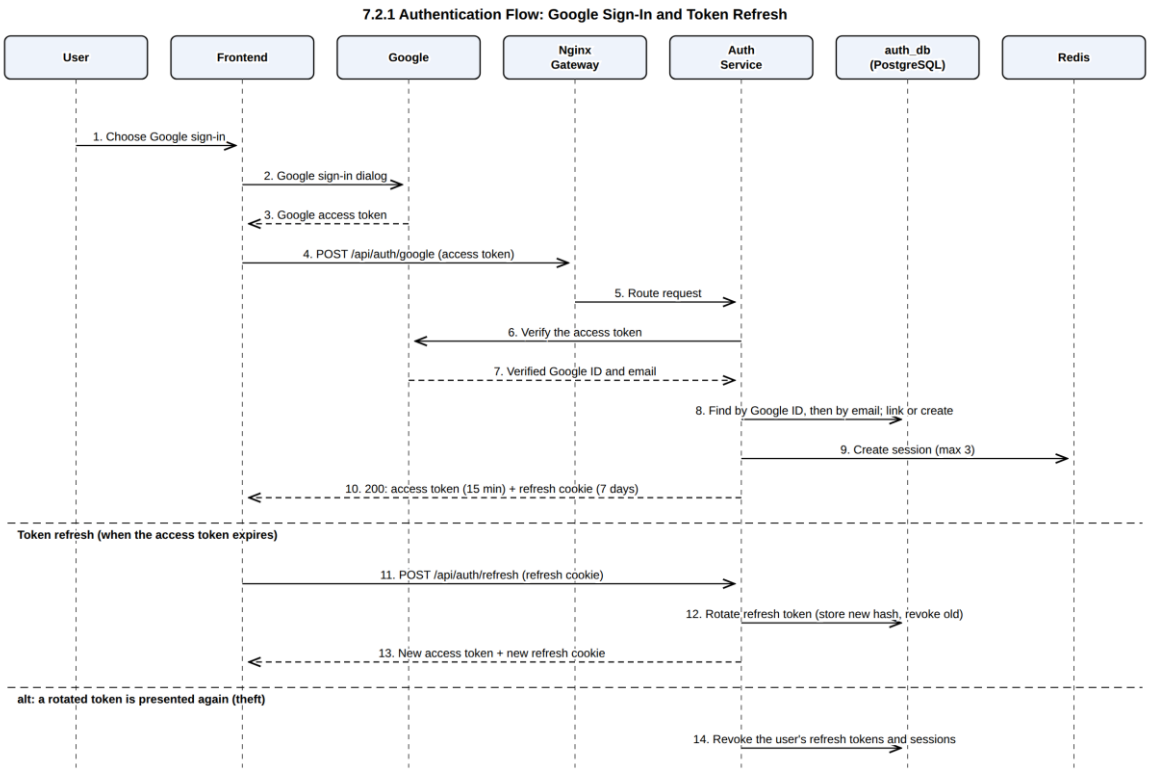


Figure 28: Authentication Flow

7.2.2 Token Management

JSON Web Tokens (JWT):

- Access tokens issued as JWTs [5]
- Token structure:
    - Header: Algorithm (HS256), Token type
    - Payload: User ID (sub), Role, Email, Issued At, Expiration
    - Signature: HMAC signature using the Auth Service signing secret
- Token expiration: 15 minutes for access tokens
- Refresh token expiration: 7 days
- Tokens stored securely:
    - Access tokens: kept by the frontend in browser storage (localStorage when remember-me is chosen, otherwise sessionStorage)

- Refresh tokens: HttpOnly, Secure cookie with SameSite=Lax

Token Validation:

- Each service validates the JWT signature through shared/auth.py
- Token expiration checked on every request
- Logout revokes the refresh token and deletes the Redis session
- Refresh token rotation on each use, with detection of a reused token (treated as theft; sessions are revoked)

7.2.3 Session Management

Session Storage:

- Session data stored in Redis (no application-level encryption)
- Session key mapped to the user ID and role
- Session lifetime: 7 days; there is no inactivity timeout (the access token expires after 15 minutes)
- Concurrent session limit: 3 active sessions per user (the oldest session is removed)

Session Security:

- Session IDs generated as random UUIDs (uuid4)
- A new session ID is created at every login
- IP address and user-agent validation is not implemented

7.2.4 Email Verification and Password Reset

Registration Flow:

- Email verification is required before login
- A signed verification link is sent to the registered email address
- Link validity: 24 hours
- A new link can be requested once every 60 seconds
- The answer to a resend request is the same for every address, so accounts cannot be probed

- The link carries a signed token that is valid only for the address it was sent to

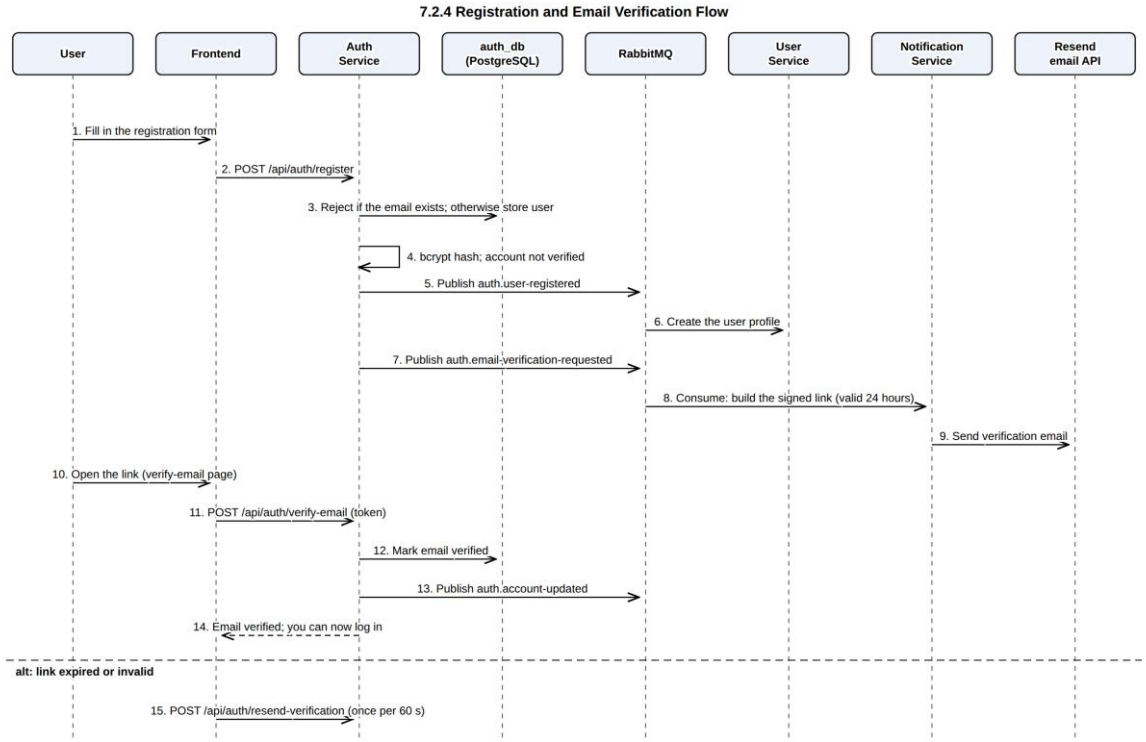


Figure 29: Registration Flow

Password Reset Flow:

- Password reset initiated via email
- A reset link is emailed (same response whether or not the address is registered)
- The signed token in the link is required to proceed
- Password reset link valid for 60 minutes
- The link is single use: it is tied to the current password hash, so it stops working once the password changes; all sessions are revoked after a reset
- An OTP service with a 5-minute Redis lifetime also exists in the Auth Service but is not used by these flows.

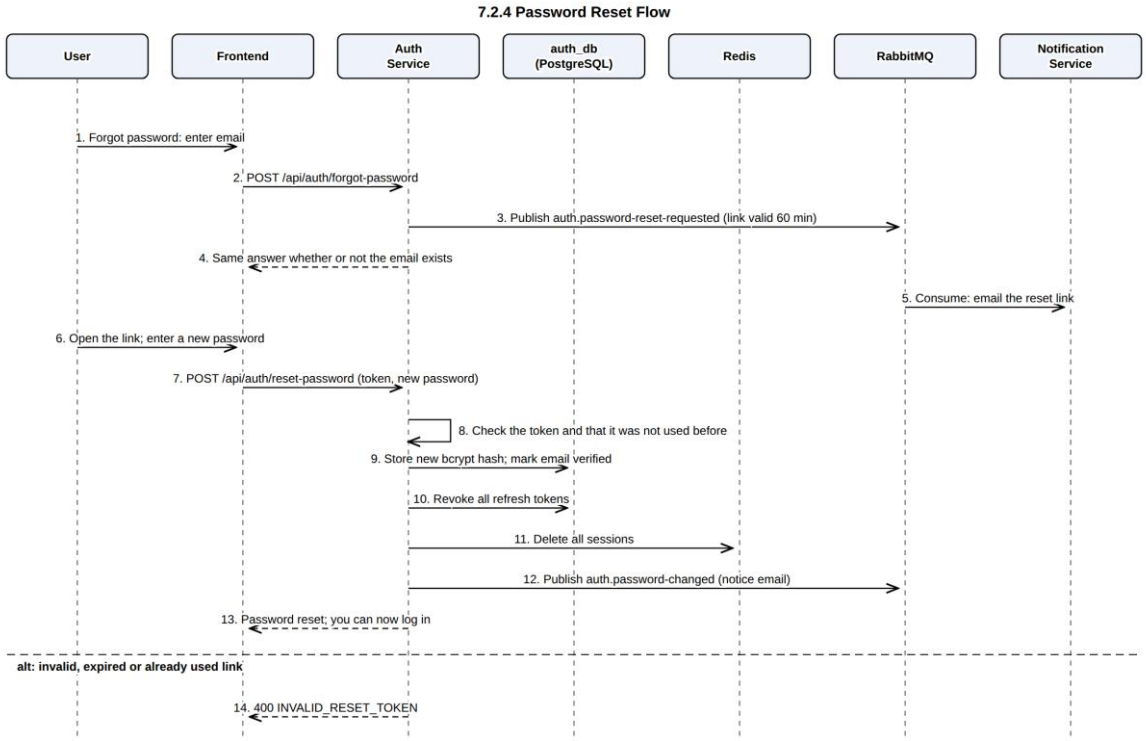


Figure 30: Password Reset Flow

7.3 Authorization Levels

7.3.1 Role-Based Access Control (RBAC)

Defined Roles:

Student Role:

- Permissions:
    - View enrolled courses
    - Access course content (videos, documents)
    - Take assessments
    - View own progress and certificate records (no certificate document is generated)
    - Upload payment receipts
    - Update own profile
- Restrictions:

- Cannot create or modify courses
- Cannot access other students' data
- Cannot approve payments
- Cannot access admin functions

Teacher Role:

- Permissions:
  - All student permissions (for enrolled courses)
  - Create, update, delete own courses
  - Create and manage assessments
  - Upload course content
  - View enrolled students' progress in own courses
  - Generate reports for own courses
  - Respond to student queries
- Restrictions:
  - Cannot access other teachers' courses (unless explicitly shared)
  - Cannot approve payments
  - Cannot manage users
  - Cannot access system-wide analytics

Administrator Role:

- Permissions:
  - All teacher permissions
  - Manage all users (create, update, deactivate, delete)
  - Manage all courses regardless of creator
  - Approve/reject manual payment receipts

    - Access system-wide analytics and reports
    - Configure system settings
    - Change user roles
    - View audit logs
    - Access all data across the platform
- Restrictions:
    - No restrictions (full system access)

7.3.2 Permission Enforcement

API Level Authorization:

- Every API endpoint protected by authorization middleware
- JWT payload contains the user role
- require_role validates the role before the request is processed
- Unauthorized requests return 403 Forbidden response

Data Level Authorization:

- Ownership checks in the service code (for example assert_course_owner) restrict access to own data; database row-level security is not used
- Users can only access data they own or have permission to view
- Example: Student can only view their own enrollment records
- Example: Teacher can only modify their own courses

Resource Ownership Validation:

- Object ownership checked before modification operations
- Example: User can only update their own profile
- Example: Teacher can only delete their own course content

7.4 Data Protection Approach

7.4.1 Data Classification

Sensitive Data:

- User credentials (passwords, tokens)
- Personal identifiable information (PII): Names, email, contact numbers, date of birth
- Payment information: Card details (handled by PayHere, not stored), transaction records
- Assessment answers and scores

Confidential Data:

- Course content (proprietary to instructors)
- Student progress and performance data
- System configuration and credentials

Public Data:

- Published course listings
- Public user profiles (if applicable)
- General platform information

7.4.2 Data Storage Security

Database Encryption:

- Data at rest encryption is provided by Neon (managed by the provider)
- Backups are managed by the database provider; the project has no backup job
- Encryption keys are managed by the provider

Field-Level Encryption:

- Field-level encryption (AES-256-GCM) was designed but is not implemented

Password Security:

- Passwords never stored in plaintext

- Hashing algorithm: bcrypt with salt
- Work factor: 12 rounds
- Password complexity requirements enforced:
    - Minimum 8 characters
    - At least one uppercase letter
    - At least one lowercase letter
    - At least one number
    - At least one special character

7.4.3 Data Transmission Security

Transport Layer Security (TLS):

- All communication over HTTPS
- TLS is terminated by the Caddy edge proxy (TLS 1.2 and 1.3 defaults)
- Cipher suites are Caddy's defaults; no custom configuration
- Certificate validation required
- HTTP Strict Transport Security (HSTS) enabled

API Security:

- Protected endpoints require a bearer token; 42 of 174 endpoints are public by design
- No payload encryption beyond TLS
- API rate limiting to prevent abuse
- CORS policy configured to allow only trusted origins

File Uploads:

- File upload size limits enforced
- File type validation (whitelist approach)
- Virus scanning is not implemented
- Uploaded files stored in isolated storage (Cloudinary)

- Uploaded media is delivered through signed, expiring URLs

7.5 Input Validation and Sanitization Strategy

7.5.1 Client-Side Validation

Form Validation:

- Real-time validation feedback for user inputs
- Type checking (email format, numeric values, date formats)
- Length constraints (minimum/maximum characters)
- Required field validation
- Custom validators for complex fields (e.g., strong password check)

JavaScript Validation:

- Input sanitization before display to prevent XSS
- HTML encoding of user-generated content
- Removal of potentially malicious scripts

Note: Client-side validation is for user experience only; all validation is re-enforced server-side.

7.5.2 Server-Side Validation

Input Validation:

- All inputs validated against expected data types and formats
- Length constraints enforced
- Range validation for numeric inputs
- Whitelist validation for enumerated values
- Regular expression validation for patterns (email, phone, etc.)

SQL Injection Prevention [6]:

- Parameterized queries used for all database operations
- ORM (SQLAlchemy) used to abstract database interactions

- No dynamic SQL query construction with user input
- Stored procedures are not used

Cross-Site Scripting (XSS) Prevention [6]:

- All user-generated content escaped before rendering
- A Content-Security-Policy header is not set (accepted limitation, ADR-17)
- HttpOnly and Secure flags on cookies
- React escapes rendered content by default; no server-side HTML sanitisation library is used

Cross-Site Request Forgery (CSRF) Prevention:

- CSRF tokens are not used: state-changing requests need a bearer token in the Authorization header, which a cross-site form cannot send
- The refresh cookie is Secure and HttpOnly
- SameSite cookie attribute set to prevent CSRF

Command Injection Prevention:

- Avoid executing system commands with user input
- If necessary, strict whitelisting and escaping
- Use library functions instead of shell commands where possible

7.5.3 File Upload Validation

File Type Validation:

- MIME type checking
- File extension whitelist (PDF, DOCX, PNG, JPG, MP4, etc.)
- Magic number validation to prevent extension spoofing
- Content scanning for malicious code is not implemented

File Size Limits:

- Video files are not uploaded: lectures are linked from YouTube or Cloudinary

- Documents: Maximum 25 MB (course materials 20 MB)
- Avatars and images: Maximum 5 MB
- Receipt images: Maximum 2MB

Upload Security:

- Files uploaded to isolated storage (Cloudinary)
- Original filenames sanitized
- Unique identifiers assigned to uploaded files
- No virus or malware scanning
- Access to files controlled through signed URLs

7.6 Encryption Standards

7.6.1 Data at Rest Encryption

Database Encryption:

- Encryption at rest provided by Neon
- Algorithm and keys managed by the provider

File Storage Encryption:

- Cloudinary storage security is managed by the provider
- Encryption at storage provider level
- Access through signed URLs and API credentials

Backup Encryption:

- Backups are managed by the database provider
- Backup encryption and retention have not been verified

7.6.2 Data in Transit Encryption

TLS Configuration:

- Caddy defaults (TLS 1.2 and 1.3)
- TLS 1.3 preferred

- Cipher suites: Caddy defaults
- Weak ciphers disabled (RC4, DES, MD5)
- Perfect Forward Secrecy (PFS) enabled

Certificate Management:

- SSL/TLS certificates from trusted Certificate Authority
- Automated certificate renewal (e.g., Let's Encrypt)
- Certificate pinning for mobile applications (future)

Inter-Service Communication:

- Microservices communicate over internal network
- Mutual TLS (mTLS) between services is not implemented; traffic stays inside the cluster network (no network policies, ADR-17)
- The Caddy edge proxy enforces HTTPS for all external traffic

7.6.3 Encryption Key Management

Key Storage:

- Secrets are stored as GitHub environment secrets and injected into Kubernetes Secrets at deploy time; Azure Key Vault is not used
- Keys never hardcoded in application code
- Environment variables used for key references
- Principle of least privilege for key access

Key Rotation:

- Automatic key rotation is not implemented

Key Hierarchy:

- No key hierarchy is used (field-level encryption is not implemented)

CHAPTER 8 DEPLOYMENT DESIGN

8.1 Hosting Plan & Infrastructure

The staging environment is hosted on Microsoft Azure Kubernetes Service. The database, media delivery, email and sign-in are provided by external services.

- **Azure Kubernetes Service (AKS):** Runs the frontend, gateway, eleven services, Redis and RabbitMQ as Deployments on one Standard_D2als_v7 node (2 vCPU, 4 GB); self-healing is provided by readiness, liveness and startup probes.
- **PostgreSQL Neon:** A serverless, managed database that scales compute resources independently of storage.
- **Redis (in cluster):** Runs as a workload inside AKS to reduce cost (ADR-13); it holds short-lived data only.
- **Cloudinary and YouTube:** External providers for media delivery and video hosting.

8.2 Hardware & Resource Requirements

Since the infrastructure is cloud-based, "hardware" refers to the virtual machine (VM) specifications allocated within Azure node pools.

Table 1: Hardware and Resource Requirements

Component	Azure VM Size	vCPUs	RAM	Role
AKS node (staging)	Standard_D2als_v7	2	4 GB	Frontend, gateway, 11 services, Redis, RabbitMQ, Caddy
Database (Neon)	Serverless	Provider-managed	Provider-managed	Eleven PostgreSQL databases
Media providers	External	n/a	n/a	Cloudinary and YouTube
Email, sign-in and payment	External	n/a	n/a	Resend, Google, PayHere (sandbox)

8.3 Network Diagram

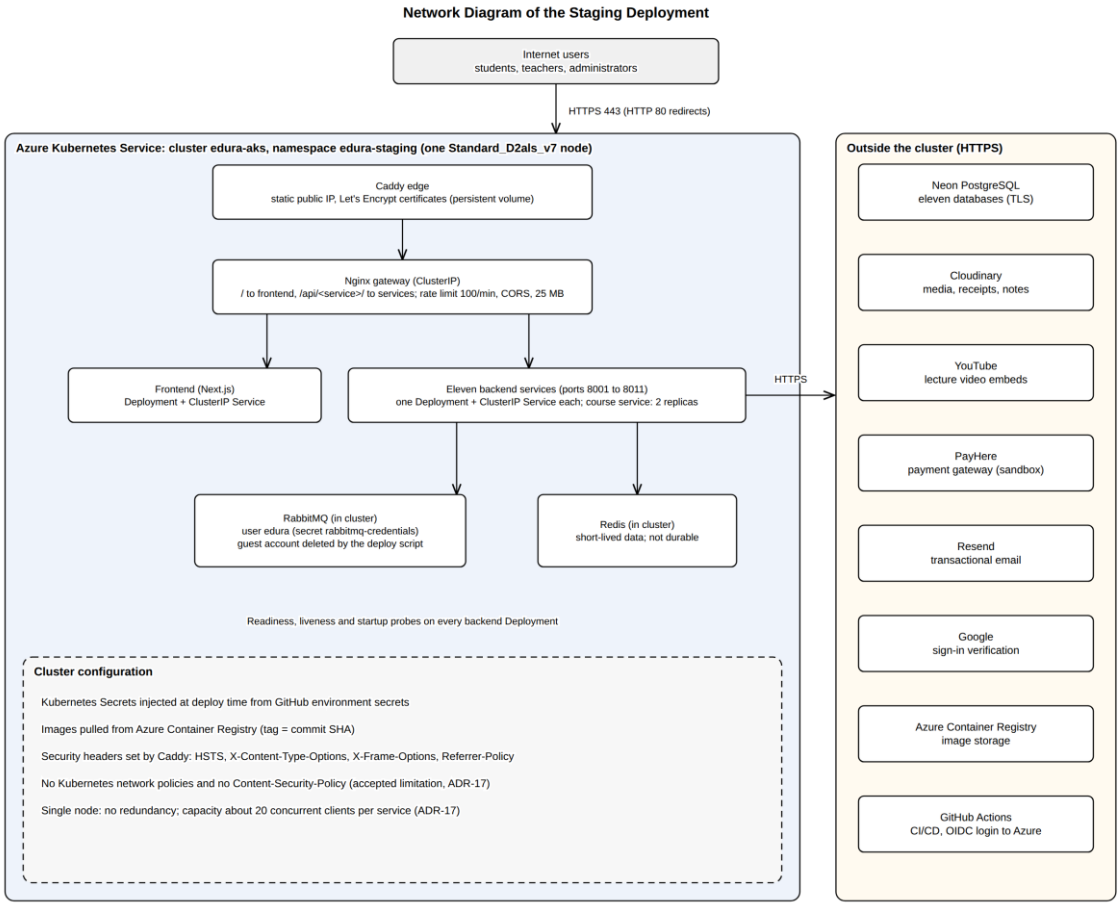


Figure 31: Network diagram

8.4 Environment Specifications

The deployment is split across three distinct environments to maintain code quality and stability.

- **Local development:** A Docker Compose stack of the eleven services, PostgreSQL 16, Redis 7, RabbitMQ 3.12 and Nginx; the frontend runs through npm run dev.
- **Staging:** Azure Kubernetes Service (namespace edura-staging), deployed automatically on merge to develop and checked by a smoke test; the cluster was paused during part of development to conserve credits (ADR-13).
- **Production:** Deployed from main through a workflow with manual approval. Multi-zone redundancy and geo-replicated backups were not implemented, and the 99.9% uptime target is not measured.

8.5 CI/CD Pipeline Overview

EDURA uses GitHub Actions to automate the path from code commit to deployment.

- **Continuous Integration (CI):** On every push and pull request the pipeline runs, for every backend service, Ruff and Black, pip-audit, pytest with coverage and an Alembic migration cycle; frontend lint, build, unit tests and npm audit; and infrastructure checks (RabbitMQ topology idempotency, gateway configuration tests, a Docker Compose smoke test). Images are built and pushed to Azure Container Registry, tagged with the commit SHA.
- **Continuous Deployment (CD):** Staging is deployed automatically on a merge to develop and is followed by a smoke test.
    - **Production:** Requires manual approval. Canary deployment was not implemented.

CHAPTER 9 CHANGES FROM VERSION 1.0

The differences between the design approved in version 1.0 and the system that was implemented are listed below. Each change arose from an implementation constraint, a security or cost decision, or the time available.

Table 2: Changes from Version 1.0

#	Version 1.0 design	Final design	Reason
1. Identity provider	Asgardeo (OAuth 2.0 / OIDC) with RS256 tokens validated at the gateway.	The Auth Service issues HS256 JWTs; every service validates them through shared/auth.py; optional Google sign-in.	Less external dependency; role enforcement inside every service.
2. Service communication	Asynchronous messaging and synchronous REST.	Asynchronous events only on a RabbitMQ topic exchange, with local replicas.	Synchronous calls coupled the availability of services (ADR-12).

3. Gateway	Nginx validates tokens and terminates TLS.	Nginx routes, rate-limits and sets CORS; Caddy terminates TLS; services authenticate requests.	Separation of concerns; automatic certificates.
4. Redis	Azure Cache for Redis.	Redis running inside the cluster.	Cost (ADR-13).
5. CI/CD	Azure DevOps with canary production deployment.	GitHub Actions with manual approval before production.	Course guideline and GitHub hosting.
6. Quality and security scanning	Pylint, Snyk and Trivy.	Ruff, Black, ESLint, pip-audit and npm audit.	Tooling aligned with the Python and Node toolchains used.
7. AI assistant	Google Gemini assistant (Section 5.2.7 and Figure 10 of version 1.0).	Not implemented; a support chat to administrators exists.	API cost and time.
8. Notifications	Email and SMS gateway.	Resend email API; SMS not implemented.	Reliable email delivery; SMS outside the delivered scope.
9. Frontend framework	Next.js 14.	Next.js 15 with React 19 (ADR-11).	High-severity advisories in Next.js 14 failed the security gate.
10. Database	MongoDB if used (SRS).	PostgreSQL on Neon throughout.	Relational integrity and ACID guarantees (ADR-02).

11. Email verification and reset	6-digit OTP valid for 10 minutes, at most 3 per hour.	Signed verification link valid for 24 hours (new link once per 60 s); password reset by an emailed single-use link valid for 60 minutes. An OTP service (5 minutes) exists but is not used.	Links are completed in one step; the OTP service was kept for later use.
12. Sessions	30-minute inactivity timeout; IP and user-agent validation.	Redis session lifetime of 7 days, at most 3 sessions (oldest removed); no IP or user-agent validation.	Simplicity within the schedule.
13. Token storage	HttpOnly cookies or memory; SameSite=Strict.	Refresh token in an HttpOnly cookie (SameSite=Lax); access token kept by the frontend in browser storage.	Implementation choice; browser storage is exposed to script injection.
14. Advanced security controls	Field-level AES-256-GCM, mutual TLS, virus scanning, CSRF tokens, CSP, key rotation, Azure Key Vault.	Not implemented. Secrets are GitHub environment secrets injected into Kubernetes Secrets; bearer tokens and SameSite cookies limit CSRF; no CSP.	Higher-risk controls were prioritised; recorded in ADR-17.
15. Upload limits	Video 500 MB, documents 25 MB, images 5 MB, receipts 2 MB.	Videos are linked, not uploaded; documents 25 MB (materials 20 MB), images 5 MB, receipts 2 MB.	Video hosting is offloaded to YouTube and Cloudinary.

16. Hosting resources	D4s_v3, D8s_v3 and E4s_v3 node pools; multi-zone production.	One Standard_D2als_v7 node for staging.	Cost; capacity is limited to about 20 concurrent clients per service (ADR-17).
17. Observability	Azure Monitor.	Probes, pipeline health checks and smoke tests; no Azure Monitor.	Not implemented.
18. Analytics storage	Materialized views.	Local replica tables, Redis cache and ReportLab PDF reports; no Excel export.	Services do not read each other's databases.
19. Class model	Student, Teacher, Class, Exam and Payment classes.	Layered modules in every service with domain classes per service (see the class diagram).	Matches the service and database-per-service design.

CHAPTER 10 REFERENCES

[1] E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970.

[2] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley, 2003.

[3] D. Hardt, Ed., "The OAuth 2.0 Authorization Framework," IETF, RFC 6749, Oct. 2012. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6749

[4] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, "OpenID Connect Core 1.0 incorporating errata set 1," OpenID Foundation, Nov. 2014. [Online]. Available: https://openid.net/specs/openid-connect-core-1_0.html

[5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519

[6] OWASP Foundation, "OWASP Application Security Verification Standard (ASVS)," version 4.0, 2019. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/

APPENDIX A ENTITY-RELATIONSHIP DIAGRAMS

The entity-relationship diagram of the database of each service, generated from the implemented models, is shown below.

A.1 Auth Service

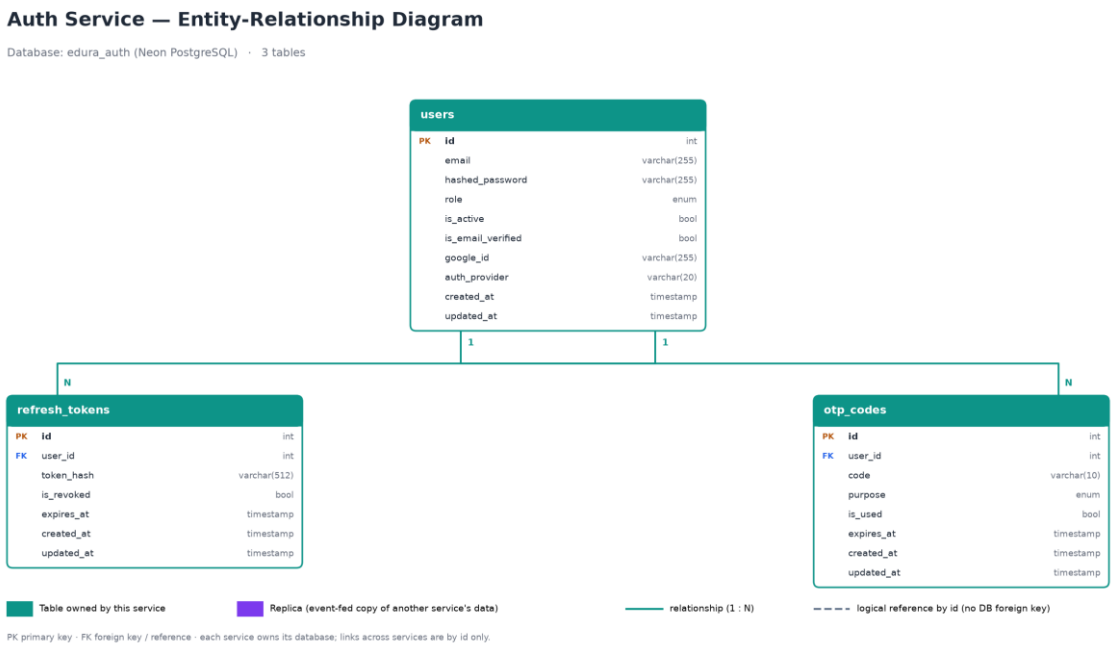


Figure A.1: Entity-Relationship Diagram of the Auth Service

A.2 User Service

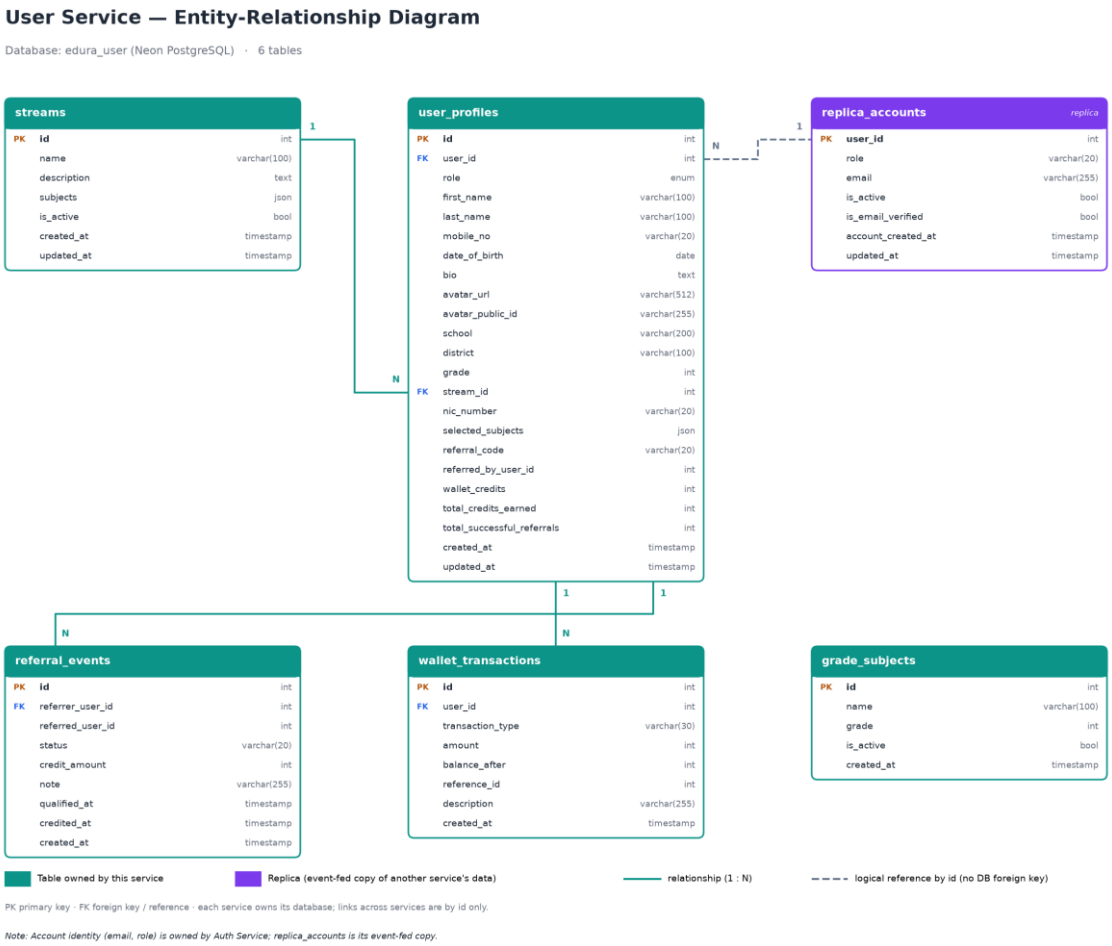


Figure A.2: Entity-Relationship Diagram of the User Service

A.3 Course Service

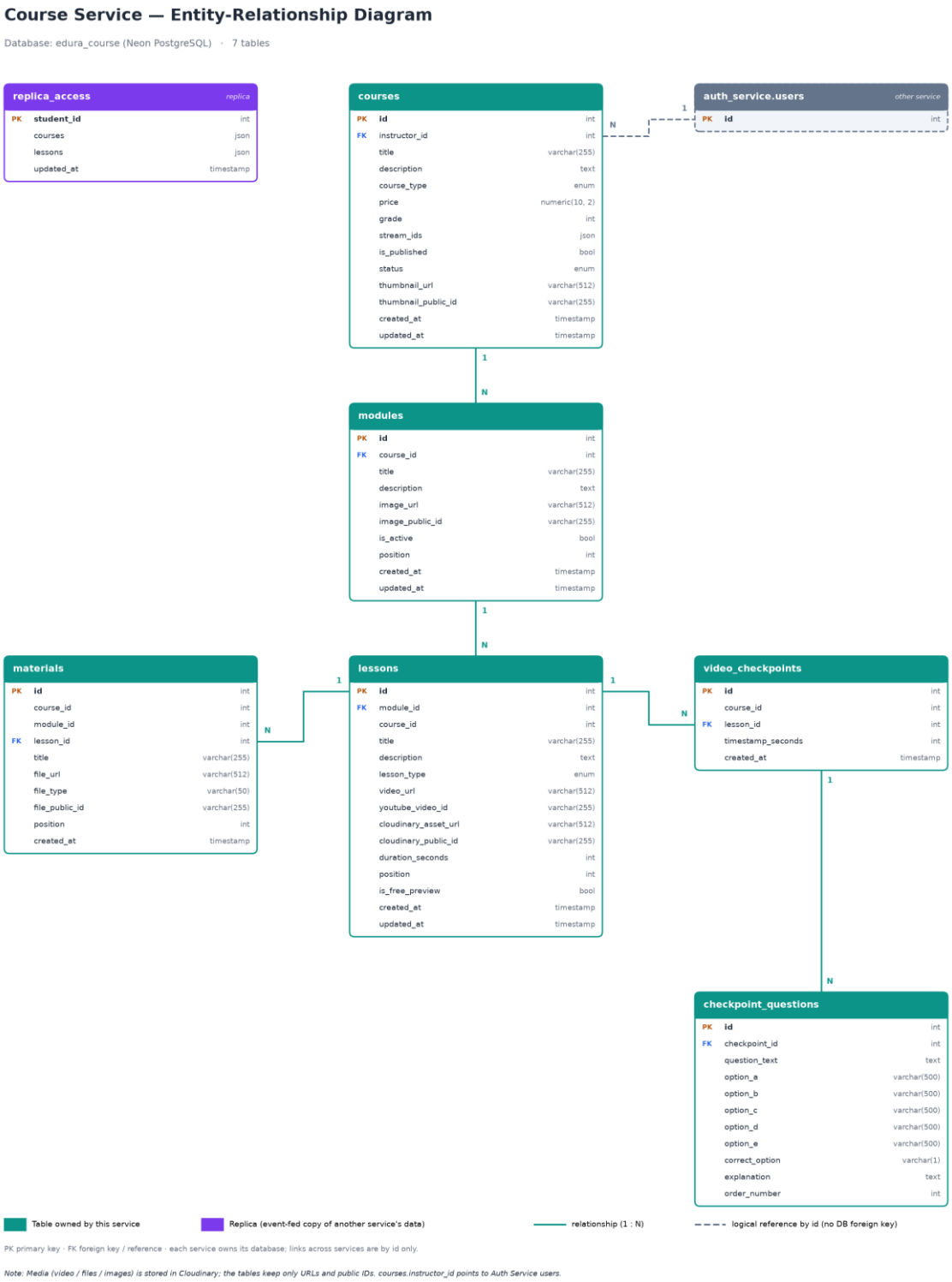


Figure A.3: Entity-Relationship Diagram of the Course Service

A.4 Content Service

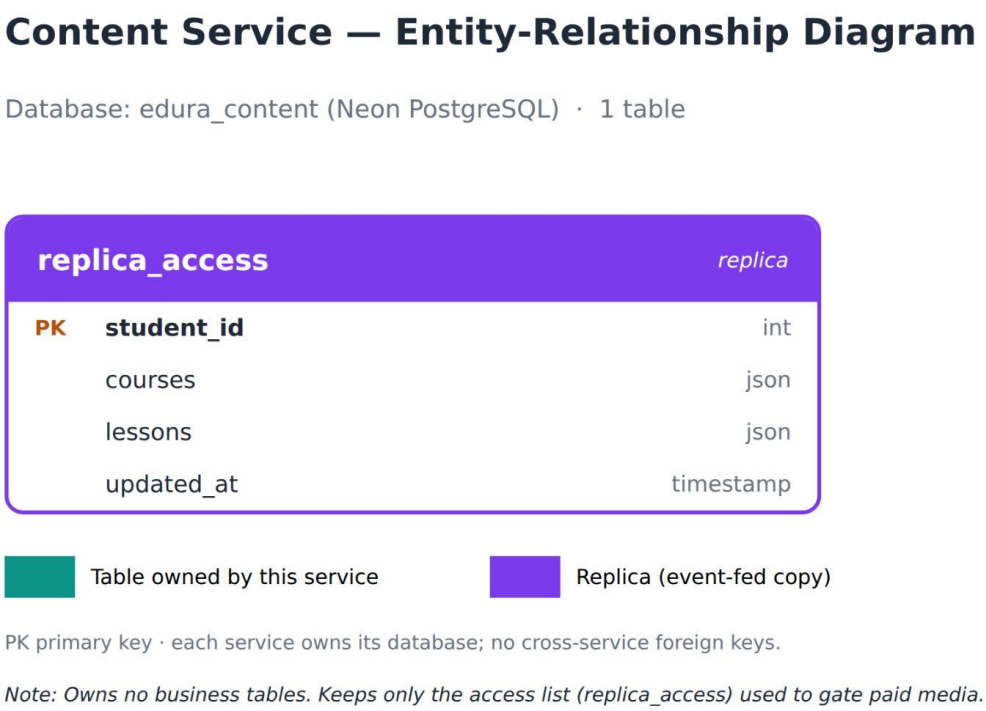


Figure A.4: Entity-Relationship Diagram of the Content Service

A.5 Enrollment Service

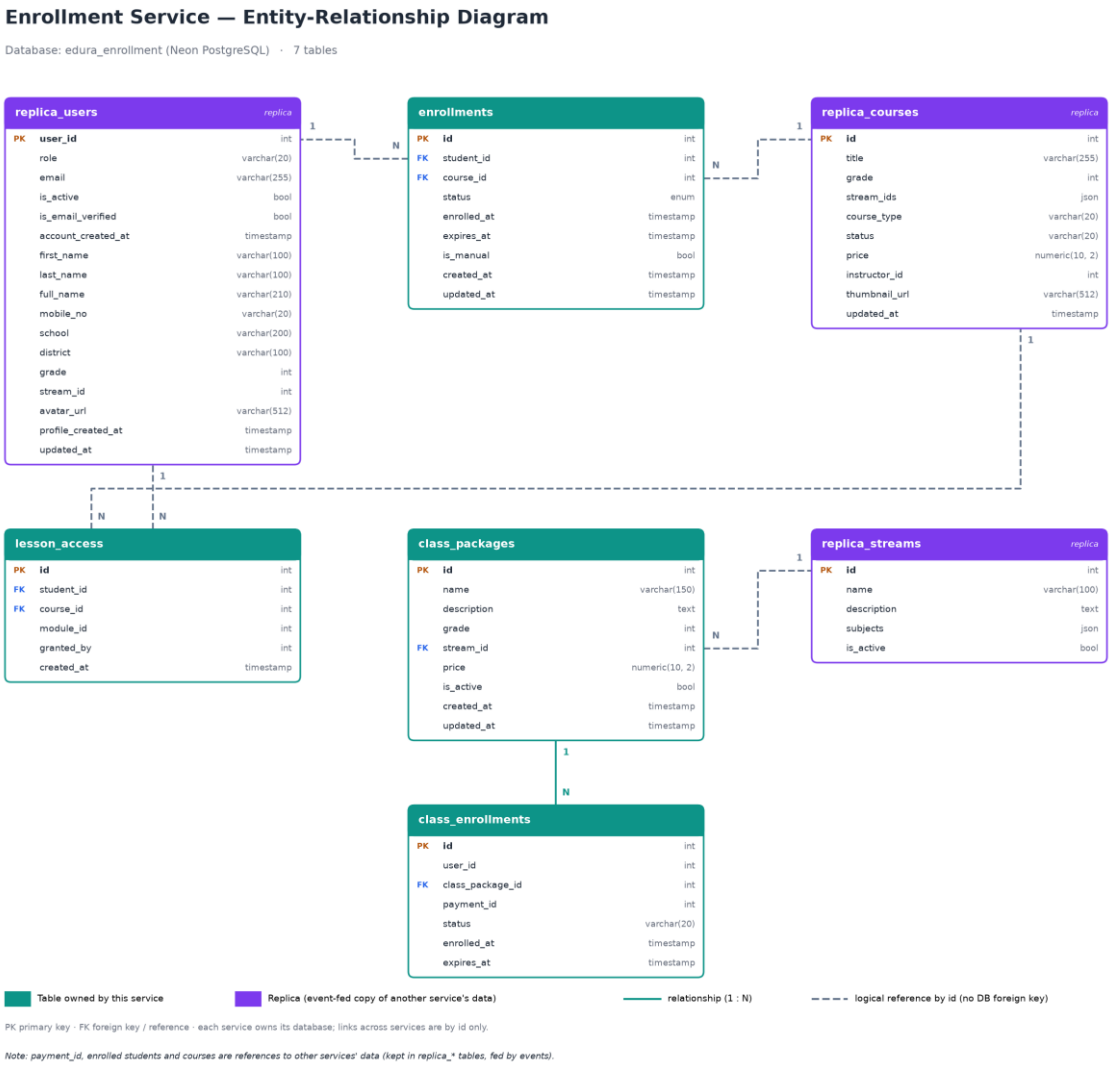


Figure A.5: Entity-Relationship Diagram of the Enrollment Service

A.6 Payment Service

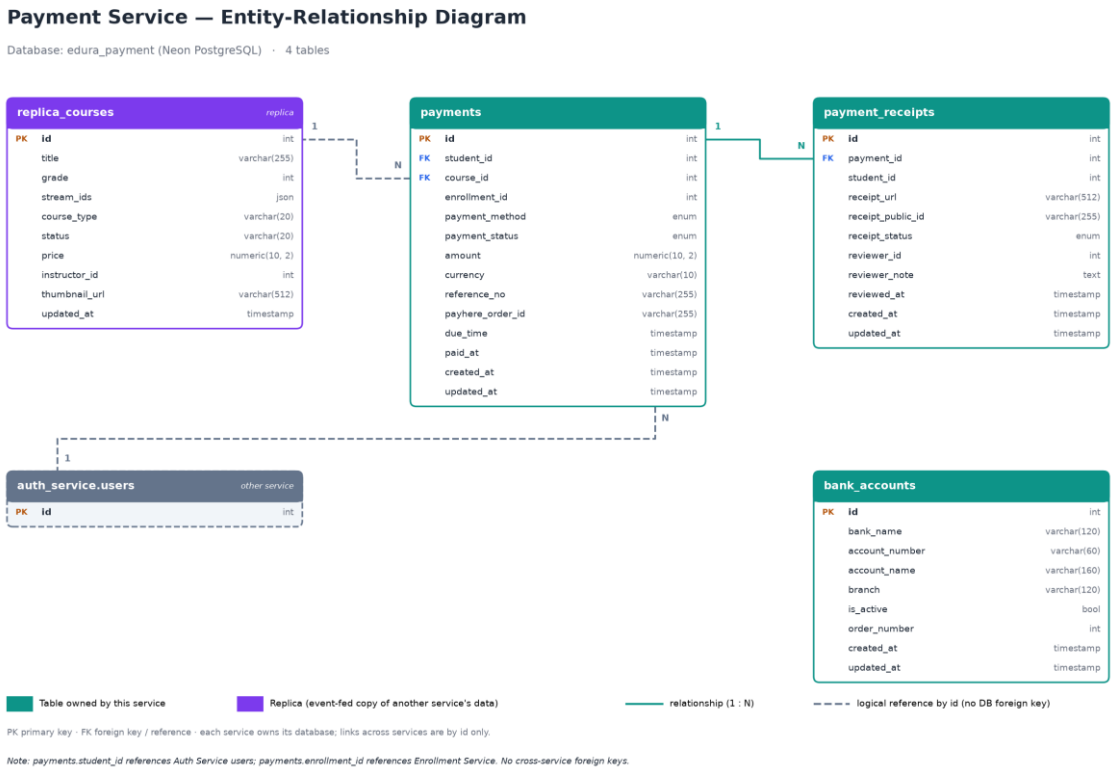


Figure A.6: Entity-Relationship Diagram of the Payment Service

A.7 Assessment Service

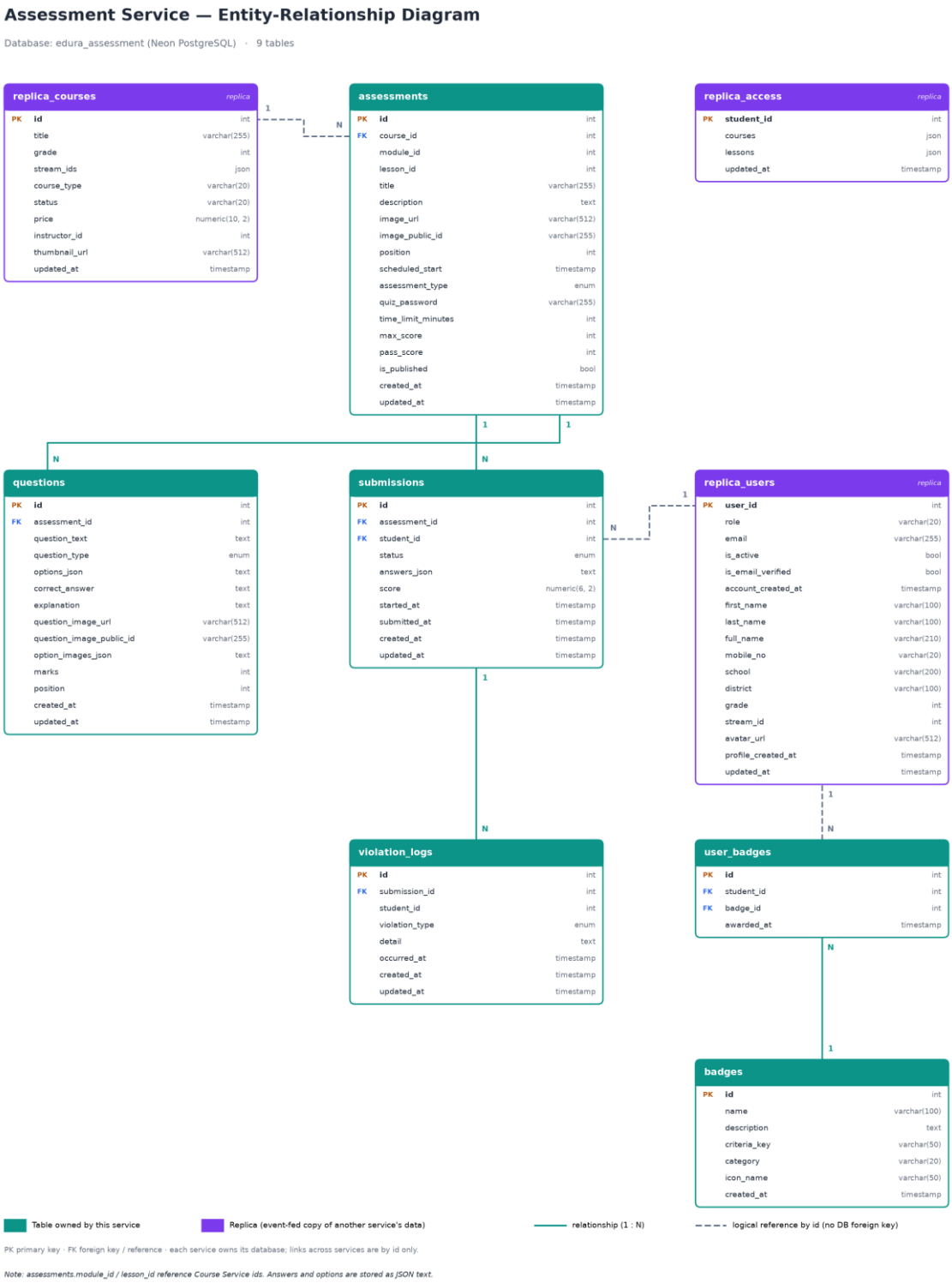


Figure A.7: Entity-Relationship Diagram of the Assessment Service

A.8 Progress Service

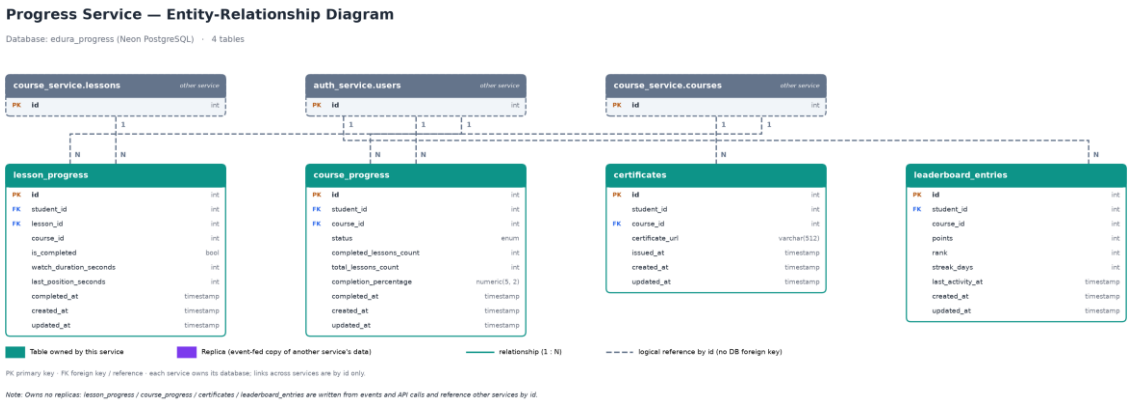


Figure A.8: Entity-Relationship Diagram of the Progress Service

A.9 Notification Service

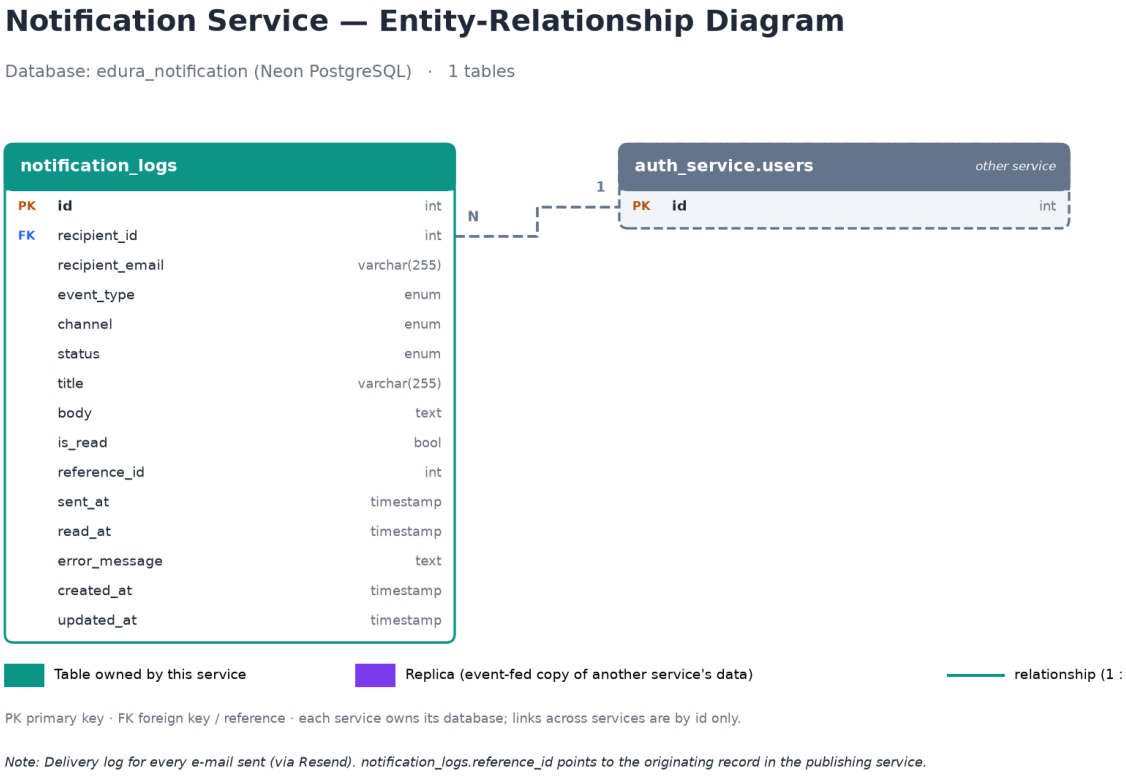


Figure A.9: Entity-Relationship Diagram of the Notification Service

A.10 Analytics Service

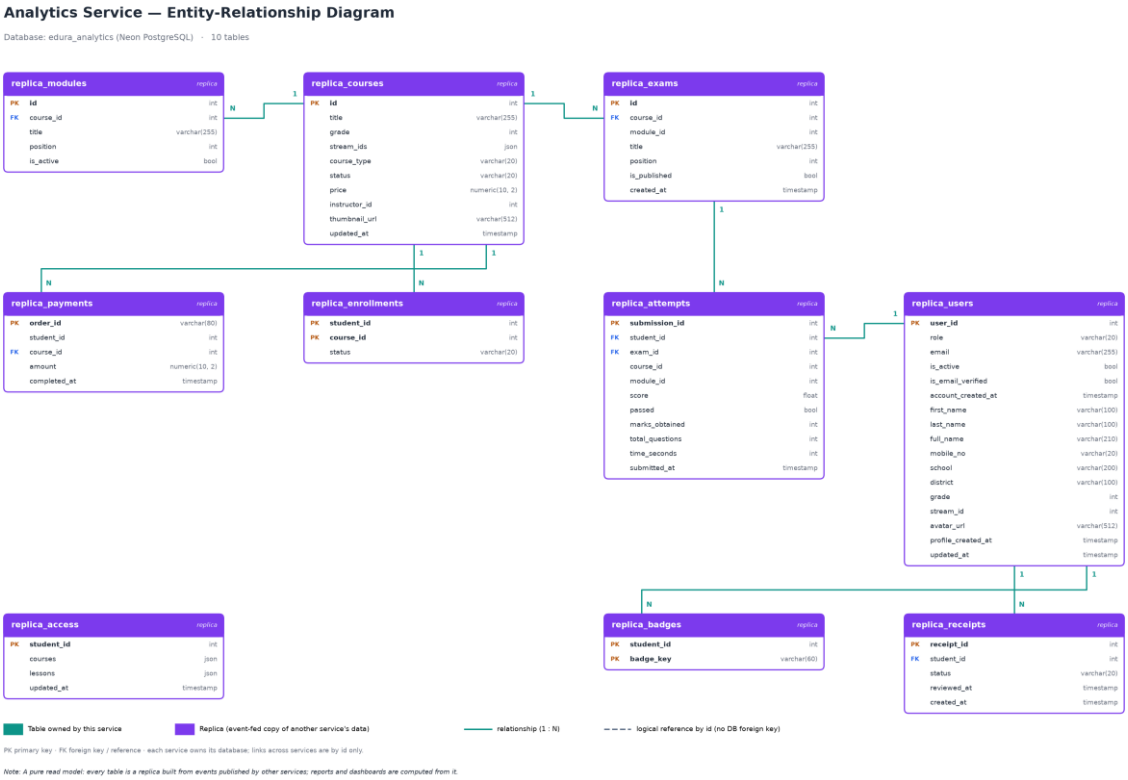


Figure A.10: Entity-Relationship Diagram of the Analytics Service

A.11 Admin Service

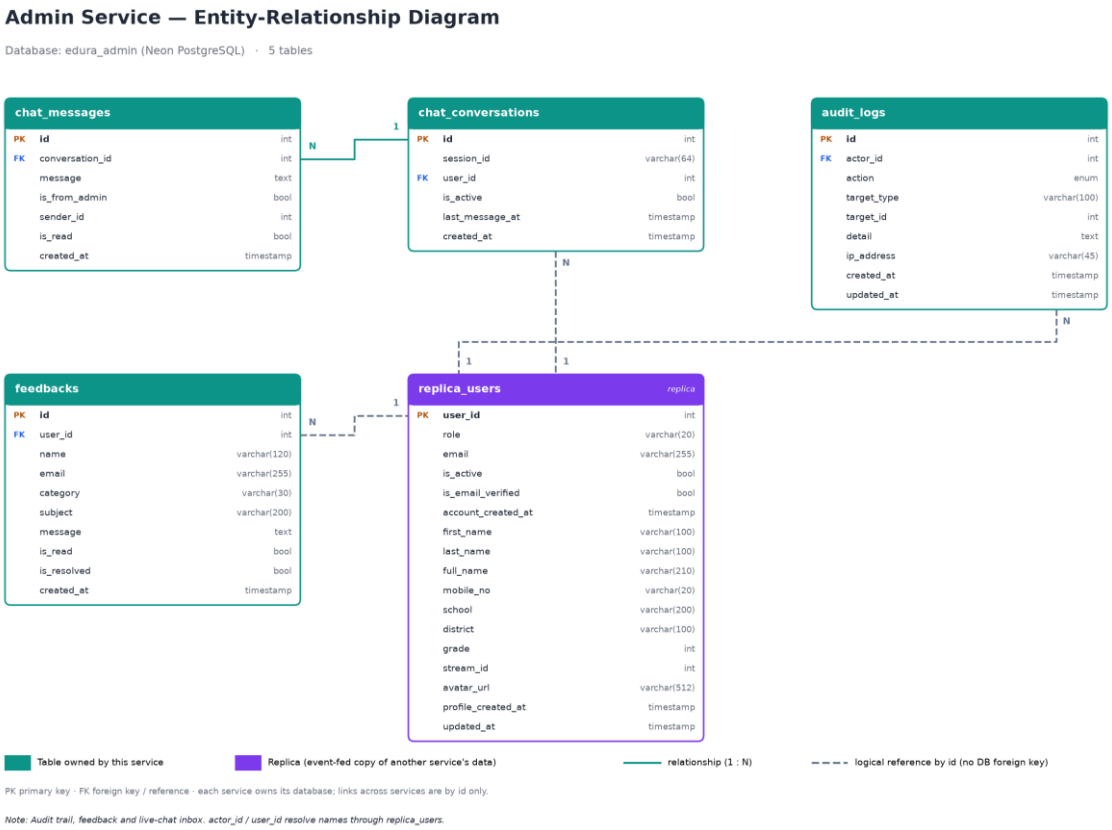


Figure A.11: Entity-Relationship Diagram of the Admin Service